

# Drone-Guided Autonomous Navigation: A Cooperative UAV–AMR System for GPS-Denied Environments

David Ortiz, Alfredo Romo, Noé Rosales, Sebastián Pulido, Jorge González

*School of Engineering and Sciences*

*Instituto Tecnológico y de Estudios Superiores de Monterrey*

*Guadalajara, México*

*A01637391@tec.mx, A01643235@tec.mx, A00227290@tec.mx, A01642759@tec.mx, A00344893@tec.mx*

**Abstract**—Many autonomous ground robots must operate where satellite positioning is unavailable or unreliable: indoor warehouses and underground facilities receive no signal, while container yards, dense urban canyons, and forest or disaster sites suffer multipath, occlusion, and blockage that degrade GPS at ground level. We present a cooperative unmanned-aerial-vehicle (UAV) and autonomous-mobile-robot (AMR) system that solves goal-directed navigation in such a GPS-denied setting. The two agents are complementary: the UAV has access to absolute positioning at altitude (provided in our testbed by a motion-capture system acting as a GPS surrogate) but cannot transport cargo, while the AMR carries no absolute-positioning instrumentation by design and therefore cannot independently determine the global location of its goal. Our pipeline has the UAV survey an unmapped  $4 \times 4$  m arena from above, build a metric occupancy map through a similarity-transform image-stitching pipeline, and localize ArUco goal markers in the world frame; this aerial prior is then handed to the AMR, which fuses it with live LiDAR sensing to plan and execute a collision-free path. Core technical contributions include a from-scratch extended Kalman filter with an adaptive visual-odometry outlier gate, a priority-weighted map-fusion rule, a frontier-exploration fallback for rejected maps, an independent automatic-emergency-braking layer, and an application-layer cryptographic middleware securing inter-robot traffic. The system is implemented entirely in ROS 2 across three networked compute platforms. In our testbed the filter reduces mean position error by  $\sim 88\%$  versus wheel odometry (to  $\sim 5$  cm), the AMR reaches the goal in 85% of trials without collision, emergency braking responds in  $\sim 0.34$  s, and a benchmark of the security layer quantifies the RSA-signing latency that motivates applying authentication selectively per topic.

**Index Terms**—GPS-denied navigation, cooperative robotics, UAV–AMR, ROS 2, extended Kalman filter, image stitching, occupancy mapping, ArUco localization, autonomous emergency braking, robot cybersecurity.

## I. INTRODUCTION

### A. Motivation and Problem Context

Modern logistics increasingly relies on autonomous ground vehicles for material transport. In structured indoor warehouses this is well understood, but a large class of industrially relevant environments—maritime ports, container yards, and partially enclosed industrial sites—combine the openness of

outdoor operation with the GPS degradation of dense infrastructure. At ground level, satellite positioning in these settings suffers from multipath reflections off metal containers and cranes, partial sky occlusion, and signal blockage, making it unreliable as the sole basis for navigation. A robot operating there cannot assume it knows where it is, nor where its destination lies, in any global frame.

This project targets exactly that regime: autonomous navigation toward a goal whose global position is not known a priori to the ground vehicle, in an environment for which no map exists in advance.

### B. Problem Statement

We consider a  $4 \times 4$  m arena populated with static obstacles and several ArUco fiducial markers that denote goal/zone locations and visual landmarks. An AMR must navigate from a start pose to a marked goal without colliding with obstacles. Two constraints define the difficulty. First, no map of the arena is available before the mission—it must be constructed at runtime. Second, the AMR carries *no* absolute-positioning instrumentation: it has wheel encoders, an inertial measurement unit (IMU), a 2D LiDAR, and a camera, but nothing that fixes its pose in a global frame. This is a deliberate modelling choice that mirrors the GPS-denied ground robot of the motivating scenario. Consequently the AMR cannot, on its own, determine the global position of a visual landmark marker it has not yet seen.

### C. Proposed Approach

We resolve the deadlock through cooperation with a second agent that *does* have absolute positioning, but at altitude. A UAV equipped with a downward camera and tracked by a motion-capture system (acting as a surrogate for the absolute positioning a real outdoor drone would obtain from GNSS) surveys the arena from above. From its georeferenced aerial video it (i) builds a metric occupancy grid of the arena and (ii) localizes the ArUco goal markers in the world frame. This aerial prior—map plus goal pose—is handed to the AMR, which fuses it with its own live LiDAR occupancy map,

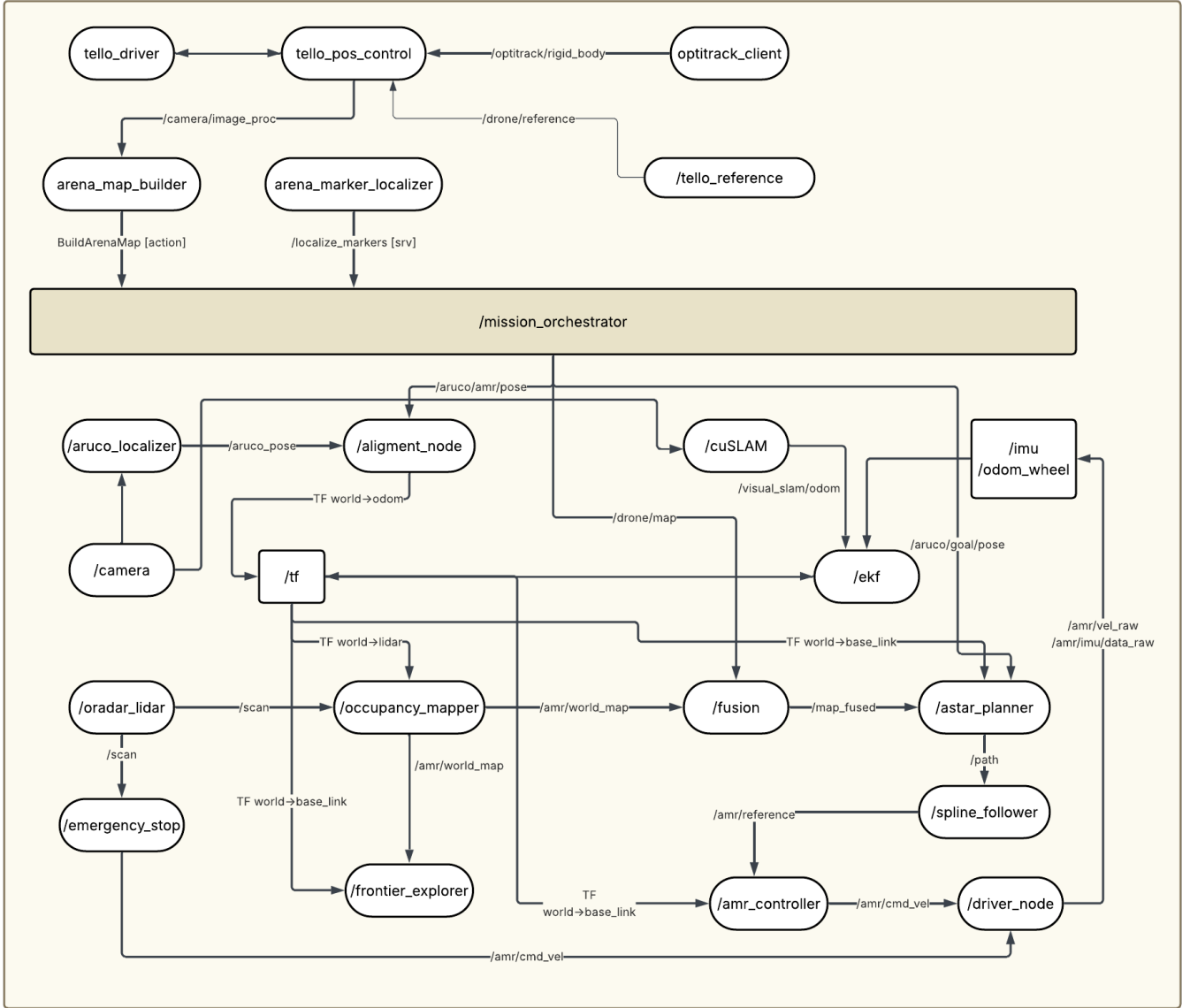


Fig. 1. Full ROS 2 computation graph of the deployed system: nodes and the topics, services, and actions connecting them across the UAV, Jetson, and RDK X3 hosts.

plans a path with A\*, and tracks that path with a feedback-linearizing controller. Throughout, the AMR maintains a local pose estimate from a custom extended Kalman filter (EKF) that fuses wheel, IMU, and visual-odometry velocity, re-anchored when goal markers are observed. The testbed in which the two robots cooperate is shown in Fig. 2.

#### D. Contributions

The system integrates a number of subsystems, each documented in depth in this report:

- A UAV position controller and scan-trajectory generator that tracks a boustrophedon survey pattern under motion-capture feedback, with a sliding-window fault detector robust to intermittent tracking dropouts (Sec. VI).
- An aerial image-stitching pipeline that builds a metric occupancy map using a similarity-transform formulation specifically chosen to remain stable over the arena’s repetitive grid texture, with an embedded deep-learning feature front-end and a learned map-quality classifier (Sec. VII).
- An aerial marker-localization subsystem combining planar PnP through a 45° mirror rig with robust geometric-median aggregation and a calibrated bias model (Sec. VIII).
- A from-scratch EKF that fuses wheel, IMU, and gated cuVSLAM velocity with absolute ArUco wall-marker fixes, using a two-stage adaptive gate to reject the velocity bursts produced by visual-SLAM loop closures (Sec. IX).
- A priority-weighted map-fusion rule that merges the

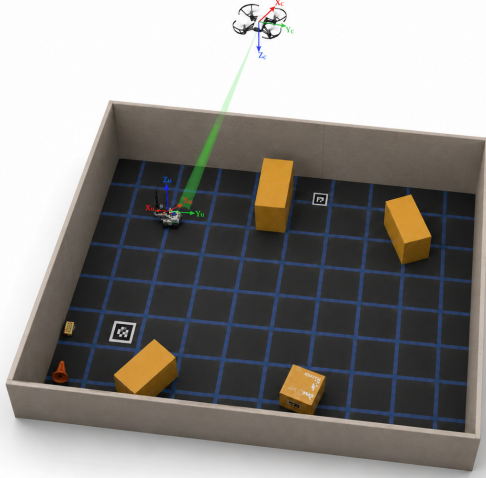


Fig. 2. Overview of the test arena with both robots: the AMR on the floor and the UAV surveying from above, amid cardboard-box obstacles and ArUco markers.

aerial prior with the AMR’s live map without any registration step (Sec. XII), and a frontier-exploration fallback that recovers the mission autonomously when the aerial map is rejected (Sec. XIV).

- An independent, high-rate automatic-emergency-braking layer (Sec. XV) and an application-layer security middleware whose cryptographic cost we benchmark to justify selective, per-topic deployment (Sec. XVI).

## E. Document Organization

Section II provides theoretical background. Section III states system requirements, and Section IV presents the hardware and software architecture. Section V describes the cooperative mission pipeline that frames the subsystem deep dives in Sections VI–XVI. Section XVII reports experimental results, and Section XVIII concludes.

## II. THEORETICAL BACKGROUND

This section collects the theory the subsystems rely on. Each primer is kept brief; full derivations and the design choices specific to our implementation appear in the corresponding deep-dive section.

### A. Differential-Drive Kinematics and Feedback Linearization

A differential-drive (unicycle) robot has configuration  $(x, y, \theta)$  evolving as  $\dot{x} = v \cos \theta$ ,  $\dot{y} = v \sin \theta$ ,  $\dot{\theta} = \omega$ , where  $v, \omega$  are body-frame forward and angular velocity. The robot is non-holonomic: it cannot translate instantaneously along its lateral axis. This makes direct proportional control of the geometric centre ill-posed, because the mapping from  $(v, \omega)$  to centre velocity is singular. Feedback linearization sidesteps this by controlling a *virtual point* offset a fixed distance  $R$  ahead of the centre; the map from  $(v, \omega)$  to that point’s velocity is invertible for all  $\theta$ , yielding linear, decoupled error dynamics (Sec. X).

### B. The Extended Kalman Filter

The EKF [2], [3] estimates the state of a nonlinear system by alternating a *prediction* step, which propagates the state mean through the nonlinear process model  $f$  and the covariance through its Jacobian  $F = \partial f / \partial \mathbf{X}$ , with *update* steps that correct the prediction using measurements through an observation model—linear in our case, so each update reduces to the ordinary Kalman correction. Each update computes a Kalman gain trading off process and measurement uncertainty. Numerically robust implementations use the *Joseph form* of the covariance update, which preserves symmetry and positive semi-definiteness regardless of gain magnitude (Sec. IX).

### C. Bayesian Occupancy Grids

An occupancy grid [2] discretizes the world into cells, each holding the probability that it is occupied. Maintaining this in *log-odds* form,  $l = \log \frac{p}{1-p}$ , turns Bayesian measurement integration into simple addition: each sensor reading adds an occupied or free increment to the cells it observes, with the cells along a range beam updated by ray casting (Bresenham). Asymmetric increments encode that a single occupied return is more informative than a single free return (Sec. XI).

### D. Feature-Based Image Stitching

Stitching aligns overlapping images by detecting keypoints (e.g. SIFT [4]), matching their descriptors, and estimating a geometric transform between frames. Spurious matches are filtered by Lowe’s ratio test and by RANSAC [5], which fits the transform to the largest consistent inlier set. The transform family matters: a full projective homography has 8 degrees of freedom, whereas a *similarity* transform (translation, rotation, uniform scale) has only 4. For a nadir camera over a planar scene the true motion is a similarity, and constraining the estimate to that family prevents the extra projective degrees of freedom from fitting noise—critical over repetitive texture (Sec. VII).

### E. Marker-Based Pose Estimation

ArUco markers [7] are square fiducials whose four corners, detected in the image, provide 2D–3D correspondences for a Perspective- $n$ -Point (PnP) pose solve. For a planar target the problem reduces to a homography from which the pose is recovered in closed form; the Infinitesimal Plane-based Pose Estimation (IPPE\_SQUARE) method [8] returns the two candidate poses of the planar ambiguity, disambiguated by reprojection error (Sec. VIII).

### F. Graph Search and Trajectory Generation

A\* [12] searches a grid graph for a minimum-cost path using an admissible heuristic; on an 8-connected grid the octile distance is the tight admissible heuristic. The resulting waypoint sequence is converted to a smooth, time-parameterized trajectory by fitting an arc-length-parameterized cubic spline and applying a trapezoidal speed profile, yielding continuous reference position and velocity for the tracking controller (Sec. XIII).

### G. Applied Cryptography Primitives

Message authentication and confidentiality rest on three primitives. A public-key infrastructure (PKI) binds identities to public keys through certificates signed by a trusted certificate authority (CA). RSA-PSS produces asymmetric signatures giving integrity and sender authenticity. AES-256-GCM [14] is a symmetric authenticated cipher giving confidentiality. Combining them—sign then encrypt at the sender, verify before decrypt at the receiver—yields a channel that is both authenticated and confidential (Sec. XVI).

### H. Related Work

Cooperative aerial-ground teams have been studied mainly for coverage and sensing, exploiting a UAV’s vantage to complement a UGV’s payload capacity and endurance [15]; we apply the same complementarity to GPS-denied navigation, using the aerial agent to supply both an occupancy prior and world-frame goal localization to the ground robot. Localization without GNSS commonly rests on fiducial markers [7] or visual-inertial SLAM [18]; rather than choosing one, our AMR fuses wheel and IMU odometry, cuVSLAM velocity, and ArUco wall-marker fixes in a single filter (Sec. IX). When no prior map exists, frontier-based exploration [16] drives a robot toward the boundary of known space; our fallback (Sec. XIV) adapts this with heading-inflated cluster scoring and an image-space visual-servo terminal homing. Finally, securing ROS 2 traffic is usually addressed at the transport layer through DDS-Security and SROS2 [17]; we instead apply an application-layer *SecureEnvelope*, trading transport transparency for per-node, per-topic control, mixed-trust bridging, and a single global kill switch.

## III. SYSTEM REQUIREMENTS

### A. Functional Requirements

### B. Non-Functional Requirements

### C. Mechanical and Electronic Requirements

## IV. SYSTEM ARCHITECTURE

### A. Hardware Platform

The ground robot is built on a Yahboom RDK X3 differential-capable base (quad-core ARM Cortex-A53) carrying an NVIDIA Jetson Orin Nano (8 GB RAM, GPU) as a compute co-processor. Its sensors are an OraDAR MS200 2D LiDAR (360°, 15 Hz, 0.05 m to 20 m), an integrated IMU and wheel encoders read over the SunriseRobot SDK, and a forward facing Intel Realsense D435i camera used for fiducial detection and Visual Simultaneous Localization and Mapping (VSLAM). The aerial agent is a DJI Tello quadrotor with downward and forward cameras; a 45° mirror redirects its forward camera to a nadir view. Absolute positioning of the UAV is provided by an OptiTrack motion-capture system (sub-millimetre, streamed over VRPN), standing in for the GNSS a field drone would use. Table IV summarizes the platform.

Compute is split deliberately across the two on-board processors (NFR-8): the RDK X3 runs low-level motor control

TABLE I  
FUNCTIONAL REQUIREMENTS

| ID    | Requirement                                                                                                                                                 |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-1  | The AMR shall navigate from start to a marked goal in a GPS-denied arena without human intervention.                                                        |
| FR-2  | The UAV shall autonomously survey the arena and build an occupancy map prior to AMR navigation.                                                             |
| FR-3  | The system shall localize ArUco goal markers in the world frame with accuracy sufficient for planning.                                                      |
| FR-4  | The AMR shall detect and avoid static obstacles in real time.                                                                                               |
| FR-5  | The mission shall execute without a pre-built map; the map is built at runtime.                                                                             |
| FR-6  | The system shall tolerate communication latency and degrade safely on loss.                                                                                 |
| FR-7  | Mission-critical ROS 2 topics shall be authenticated, and optionally encrypted.                                                                             |
| FR-8  | The UAV shall fly an autonomous coverage (boustrophedon) survey trajectory under closed-loop position control.                                              |
| FR-9  | The system shall localize the AMR in the world frame from a drone-observed top fiducial, to align the AMR <code>odom</code> frame with <code>world</code> . |
| FR-10 | The AMR shall fuse its proprioceptive and onboard sensors (wheel, IMU, visual odometry) into a single continuous pose estimate.                             |
| FR-11 | The system shall assess aerial-map quality automatically and fall back to autonomous frontier exploration when the map is rejected.                         |
| FR-12 | The system shall halt the AMR on imminent collision independently of, and faster than, the planning loop.                                                   |
| FR-13 | The AMR shall accommodate slowly moving obstacles through continuous occupancy-map updates (degraded, non-predictive handling).                             |

TABLE II  
NON-FUNCTIONAL REQUIREMENTS

| ID     | Requirement                                                                                                                                   |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-1  | The state estimator shall be implemented from scratch (no off-the-shelf localization package).                                                |
| NFR-2  | The LiDAR occupancy map shall update at $\geq 0.5$ Hz.                                                                                        |
| NFR-3  | The planner shall replan within 500 ms of a new map.                                                                                          |
| NFR-4  | The AEB layer shall react to obstacles within $\sim 333$ ms at the deployed LiDAR rate.                                                       |
| NFR-5  | The stitching pipeline shall fit within the Jetson Orin Nano’s 8 GB memory.                                                                   |
| NFR-6  | The system shall run on ROS 2 Humble.                                                                                                         |
| NFR-7  | The EKF and tracking-control loops shall run at 100 Hz.                                                                                       |
| NFR-8  | The compute shall be decentralized across at least two independent on-board processors (RDK X3, Jetson Orin Nano).                            |
| NFR-9  | Inter-machine map and goal topics shall use latched ( <code>TRANSIENT_LOCAL</code> ) QoS so late-joining nodes receive them.                  |
| NFR-10 | The security layer shall be globally disengageable through a single kill switch without code changes.                                         |
| NFR-11 | Cryptographic overhead shall be characterized, and security applied selectively so that each secured topic stays within its messaging budget. |

and the EKF, while the Jetson Orin Nano runs the perception- and planning-heavy workloads (stitching, fusion, planning, orchestration). The LiDAR is mounted so its scan plane lies above floor irregularities yet intersects the 30 cm-tall obstacle



TABLE III  
MECHANICAL AND ELECTRONIC REQUIREMENTS

| ID   | Requirement                                                                                                |
|------|------------------------------------------------------------------------------------------------------------|
| ER-1 | The AMR LiDAR scan plane shall sit at 13 cm—above floor discontinuities, below obstacle height.            |
| ER-2 | The UAV shall view the arena at nadir via a 45° mirror redirecting its forward camera.                     |
| ER-3 | The static transform <code>base_footprint</code> → <code>lidar</code> shall be $x = 0.07$ m, $z = 0.13$ m. |
| ER-4 | The camera-to-drone transform shall be calibrated once per deployment (roll = $\pi$ , yaw = 0.0456 rad).   |

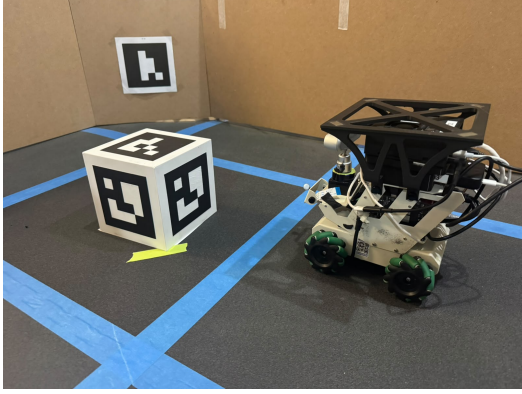


Fig. 3. The AMR with its 3D-printed top-mounted ArUco cube, which serves as the fiducial the UAV uses to localize the ground robot from above.

TABLE IV  
HARDWARE PLATFORM SUMMARY

| Component        | Specification                                    |
|------------------|--------------------------------------------------|
| AMR base         | Yahboom RDK X3, ROS 2 Humble                     |
| AMR co-processor | Jetson Orin Nano, 8 GB, GPU                      |
| LiDAR            | OraDAR MS200, 360°, 15 Hz                        |
| IMU / encoders   | Integrated, SunriseRobot SDK                     |
| AMR camera       | Intel Realsense D435i, ArUco detection and VSLAM |
| UAV              | DJI Tello, 720p, 45° mirror rig                  |
| UAV positioning  | OptiTrack (VRPN over Ethernet)                   |

boxes (ER-1, ER-3). Additionally, providing a fiducial for locating the AMR from the drone view, we equip the vehicle with a 3D-printed top-mounted base carrying an ArUco marker (Fig. 3).

### B. Software Decomposition

The software is built on ROS 2 Humble [1] (NFR-6). It spans three main repositories: the Jetson stack (`collab_nav_ground-jetson`), the RDK X3 stack (`collab_nav_ground-rasp`), and the security middleware (`security_middleware`) [20]–[22]. Functionality is decomposed into the subsystems of Table V, each realized as one or more ROS 2 packages and detailed in Sections VI–XVI. The complete ROS 2 node-and-topic graph of the deployed system is shown in Fig. 1.

TABLE V  
SUBSYSTEM-TO-PACKAGE MAPPING

| Subsystem        | Packages                                                                                                                                                                               | Host   |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| UAV flight       | <code>tello_driver</code> ,<br><code>tello_pos_control</code>                                                                                                                          | Jetson |
| Aerial mapping   | <code>arena_map_builder</code> ,<br><code>arena_marker_localizer</code>                                                                                                                | Jetson |
| AMR localization | <code>ekf_amr</code> ,<br><code>aruco_localizer</code> ,<br><code>amr_optitrack</code>                                                                                                 | RDK X3 |
| Navigation       | <code>fusion</code> , <code>world_mapper</code> ,<br><code>oradar_ros</code> ,<br><code>alignment_node</code> ,<br><code>trajectory_planner</code> ,<br><code>frontier_explorer</code> | Jetson |
| Safety & orch.   | <code>emergency_stop</code> ,<br><code>mission_orchestrator</code>                                                                                                                     | Jetson |
| Security         | <code>ros2_security</code>                                                                                                                                                             | All    |

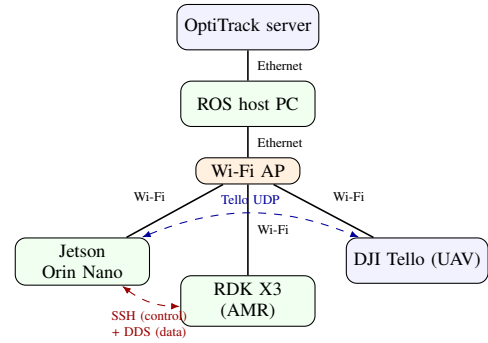


Fig. 4. Network topology. OptiTrack reaches the robot LAN over Ethernet via the ROS host and Wi-Fi access point; VRPN poses ride a secondary Wi-Fi link. The Jetson manages the RDK X3 over SSH (control plane) and exchanges topics over DDS (data plane), and bridges the Tello over its proprietary UDP SDK.

### C. Communication Topology

Two planes carry traffic. The *control plane* uses SSH (Paramiko) for the Jetson to manage the RDK X3’s lifecycle (starting and stopping the AMR service). The *data plane* is ROS 2 DDS over Wi-Fi, sharing all sensor and navigation topics across machines within a common `ROS_DOMAIN_ID`. The UAV is bridged over its proprietary Tello UDP SDK, and OptiTrack poses arrive over VRPN on a secondary Wi-Fi network. Latched (`TRANSIENT_LOCAL`) QoS is used for the map and goal topics so that late-joining nodes still receive them (NFR-9). The resulting topology is shown in Fig. 4.

### D. Development Methodology

Development followed four incremental stages: (1) basic AMR kinematics, odometry, and LiDAR obstacle detection; (2a) UAV autonomous exploration and OptiTrack positioning, in parallel with (2b) the AMR EKF, mapping, and trajectory control; (3) integration into the cooperative localization-and-handoff pipeline; and (4) full autonomous missions with robustness testing against latency, and communication loss. Although some of them developed in parallel, each stage produced a runnable increment that later stages built on.

## V. COOPERATIVE MISSION PIPELINE

The end-to-end mission is sequenced by `mission_orchestrator`, a deterministic twelve-stage finite state machine  $M = (Q, \Sigma, \delta, q_0, F)$  with  $\Sigma = \{\text{success}, \text{abort}\}$ : each stage advances only on success, and any sub-stage failure transitions to a single ABORT sink (Fig. 5). The stages are:

01. **OptiTrack bring-up.** Check `/optitrack/rigid_body` presence and header, launching the VRPN client if absent (01.a), then sanity-check its contents (frame id and freshness) (01.b).
02. **Arena map-builder bring-up.** Launch the server and configure the background-image path (02.a) and the online/offline mode (default online) (02.b). Asynchronously, the server will later return an `OccupancyGrid` and marker poses, joined at 04.b.
03. **Drone routine.** Connect to the Tello Wi-Fi (03.a), launch `tello_driver` (03.b), run pre-flight checks (live camera, battery above minimum) (03.c), launch the take-off and scanning routine (03.d), and—if online—stream processed frames to the map builder (03.e). The orchestrator then observes the flight state transitions with per-stage timeouts (03.f), waits for the recorded `scan.mp4` and telemetry (03.g), and verifies video integrity with `ffmpeg` (03.h).
04. **Aerial marker localization (concurrent with map build).** A `BuildArenaMap` goal is dispatched in the *background* while the `arena_marker_localizer` service is launched (04). The orchestrator calls `/localize_markers` as the *orientation* source, running while the map builds (04.a), then joins the map-builder result as the *position* source and publishes `/aruco/amr/pose` (position from the map builder, orientation from the localizer; an all-NaN pose if unavailable, which the alignment node reads as “no data” and falls back to the identity transform) (04.b). Finally the map-quality classifier runs on the `BuildArenaMap` diagnostic features (04.c): on PASS it publishes the `OccupancyGrid` to `/drone/map` and the goal to `/aruco/goal/pose`; on FAIL it publishes a border-template grid (no goal pose), and the frontier-exploration fallback is armed later (11.b) and triggered at 12.a.
05. **Visual SLAM.** Verify the RealSense D435i (05.a), start the container and launch Isaac ROS cuSLAM (05.b), and confirm valid tracking odometry (05.c).
06. **AMR bring-up.** Ping (06.a) and SSH into the RDK X3 (06.b), start the `amr_bringup` service and verify it is active (06.c), and wait for the IMU stream (06.d).
07. **Emergency stop.** Launch `emergency_stop` and verify `/amr/emergency_stop` is inactive (07.a).
08. **AMR marker localization.** Launch `aruco_localizer` feeding the EKF; the camera source follows the VSLAM toggle (RealSense colour when disabled, the VSLAM IR stream when

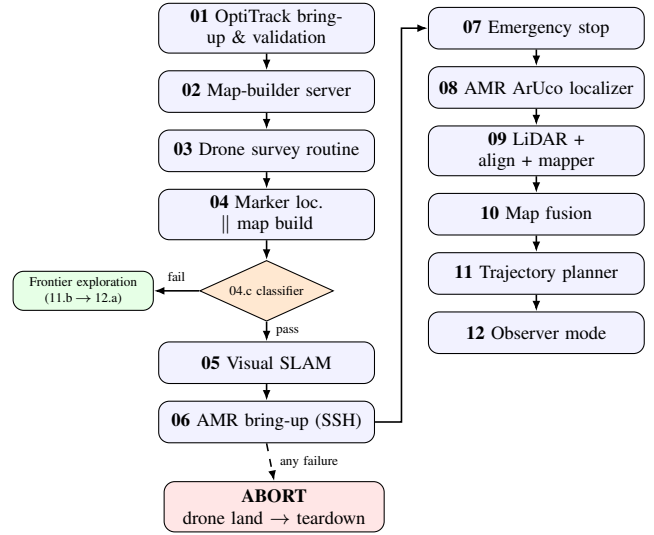


Fig. 5. Mission orchestrator state machine. Stages advance left-to-right, top-to-bottom (01–06 then 07–12). The map-quality classifier (04.c) splits the mission: on PASS the pipeline continues to goal-directed planning over `/drone/map`; on FAIL it falls back to frontier exploration over `/fusion/map` (the fallback stack is launched at 11.b and triggered at 12.a). Any sub-stage failure transitions to the ABORT sink, which lands the drone before tearing down.

enabled) (08.a).

09. **Mapping.** Launch the OraDAR LiDAR (09.a), the `alignment_node` mapping `/aruco/amr/pose` to the `world→odom` transform (09.b), and the odometry-based occupancy mapper (09.c).
10. **Map fusion.** Launch fusion overlaying the drone and AMR-built maps; fire-and-forget, as fusion only emits once `/drone/map` exists.
11. **Trajectory planner.** Launch the A\* planner and spline follower, with `map_topic = /drone/map` on PASS or `/fusion/map` on FAIL. On FAIL only, the frontier-exploration stack (Sec. XIV) is launched and idles until triggered (11.b).
12. **Observer mode.** Enter telemetry logging; if in the frontier-exploration (map FAIL) mode, send the start signal on `/map_fail_fallback/start` once bring-up completes (12.a).

Stage 04 exploits structured concurrency: map building and marker localization share inputs but are computationally independent, so pipelining them reduces wall-clock latency from  $T_{\text{stitch}} + T_{\text{PnP}}$  to  $\max(T_{\text{stitch}}, T_{\text{PnP}})$ . The classifier at 04.c is the single decision point that branches the otherwise-linear pipeline: a passing map drives goal-directed planning, while a failing map degrades gracefully to frontier exploration over the fused map rather than aborting. The abort sequence prioritizes drone safety (land command first), then tears down nodes gracefully (SIGINT→SIGKILL) and closes SSH sessions.

## VI. UAV FLIGHT CONTROL

### A. Role and Requirements

`tello_pos_control` flies the survey pattern that Section VII stitches into a map, satisfying FR-2. It must track a world-frame scan trajectory using OptiTrack pose feedback, despite the Tello exposing only a *body-frame* velocity interface for its control, and must abort safely on loss of tracking (FR-6) or direct orchestrator's command.

### B. Theory and Design Rationale

1) *Flight state machine*: Flight proceeds through six states: pre-flight, takeoff/home, stabilize at home, trajectory, return home, and normal land, with a seventh fault-land state reachable from the stabilize, trajectory, and return states when the tracking fault detector fires at detection of unphysical pose data.

2) *Position controller*: The world-frame position error is  $\mathbf{e} = \mathbf{p} - \mathbf{p}^*$ . Each axis uses PID with feedforward and anti-windup on the integral term,

$$I_{x,k} = \text{clip}(I_{x,k-1} + e_x \Delta t, -I_x^{\max}, I_x^{\max}), \quad (1)$$

$$v_x^W = -k_{p,x}e_x - k_{d,x}\tilde{v}_x - k_{i,x}I_x + v_{ff,x}, \quad (2)$$

where  $\tilde{v}_x$  is an EMA-smoothed finite-difference velocity. Because the Tello accepts body-frame commands, the world-frame command is rotated through the measured yaw  $\psi$ :

$$\begin{pmatrix} v_x^B \\ v_y^B \end{pmatrix} = \mathbf{R}(\psi)^\top \begin{pmatrix} v_x^W \\ v_y^W \end{pmatrix} = \begin{pmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{pmatrix} \begin{pmatrix} v_x^W \\ v_y^W \end{pmatrix}. \quad (3)$$

The command is then saturated in a direction-preserving manner at  $v_{\max}$ , and yaw is held with  $\omega_z = \text{clip}(-k_{p,\psi} \text{atan2}(\sin(\psi - \psi_d), \cos(\psi - \psi_d)), -1, 1)$ .

3) *OptiTrack fault detector*: Tracking can fail in an *alternating-ghost* mode, interleaving valid and corrupted samples, which defeats a naive consecutive-bad-sample counter. We instead monitor a sliding-window bad-sample ratio. With  $b_k = \mathbf{1}[\|\mathbf{p}_k - \mathbf{p}_{k-1}\| > \delta_{\text{thr}}]$ , a fault is declared (and latched) when

$$\hat{\rho}_k = \frac{1}{W} \sum_j b_j > \rho_{\text{thr}}. \quad (4)$$

The detector is active only in the stabilize, trajectory, and return states.

4) *Scan trajectory*: Survey waypoints form a boustrophedon (lawnmower) raster. The path is arc-length parameterized,  $t_i = s_i/v$  with  $s_i = \sum d_j$  the cumulative chord length, and interpolated by a clamped cubic spline with zero endpoint velocities. The analytic spline derivative supplies the feedforward velocity.

### C. Implementation

The `pos_controller_node` subscribes to the OptiTrack pose (`optitrack/rigid_body`) and a reference `Odometry`, and publishes body-frame velocity (`cmd_vel`), takeoff/land, and the integer flight state to `tello_driver`, advancing the state machine on a fixed timer (Table VI). Trajectory playback is gated by a `start_trajectory` service

TABLE VI  
UAV CONTROLLER INTERFACE

| Direction | Topic / Service                   | Type        |
|-----------|-----------------------------------|-------------|
| sub       | <code>optitrack/rigid_body</code> | PoseStamped |
| sub       | <code>reference</code>            | Odometry    |
| sub       | <code>camera/image_raw</code>     | Image       |
| pub       | <code>cmd_vel</code>              | Twist       |
| pub       | <code>drone/state</code>          | Int32       |
| pub       | <code>takeoff / land</code>       | Empty       |
| cli       | <code>start_trajectory</code>     | Empty (srv) |

TABLE VII  
UAV CONTROLLER DEPLOYED PARAMETERS

| Parameter                         | Value                 | Meaning                   |
|-----------------------------------|-----------------------|---------------------------|
| $k_{p,x}, k_{p,y}, k_{p,z}$       | 2.0, 2.0, 3.0         | position P gains          |
| $k_{d,x}, k_{d,y}, k_{d,z}$       | 0.1, 0.1, 0.3         | derivative gains          |
| $k_{i,x}, k_{i,y}, k_{i,z}$       | 0, 0, 0.1             | integral gains            |
| <code>max_speed</code>            | 0.6 m s <sup>-1</sup> | speed saturation          |
| $k_{p,\psi}$                      | 0.15                  | yaw-hold gain             |
| <code>alpha</code>                | 0.35                  | derivative EMA coeff.     |
| <code>stab_threshold</code>       | 0.10 m                | home-stabilization band   |
| <code>required_stab_counts</code> | 30                    | samples to declare stable |

TABLE VIII  
COMPLETE UAV BOUSTROPHEDON SURVEY TRACKING ERROR STATISTICS (PERCENTILES, MIN, MAX, MEAN, AND RMS) PER AXIS AND EUCLIDEAN NORM, MEASURED AGAINST OPTITRACK GROUND TRUTH OVER THE FULL SURVEY FLIGHT.

| Metric | $ e_x $ [m] | $ e_y $ [m] | $ e_z $ [m] | $\ e\ $ [m] |
|--------|-------------|-------------|-------------|-------------|
| Min    | 0.0000      | 0.0000      | 0.0000      | 0.0094      |
| P25    | 0.0197      | 0.0112      | 0.0141      | 0.0496      |
| P50    | 0.0352      | 0.0201      | 0.0277      | 0.0605      |
| P75    | 0.0539      | 0.0320      | 0.0433      | 0.0707      |
| P90    | 0.0665      | 0.0477      | 0.0587      | 0.0846      |
| P95    | 0.0763      | 0.0554      | 0.0667      | 0.0943      |
| Max    | 0.1429      | 0.0763      | 0.1058      | 0.1502      |
| Mean   | 0.0375      | 0.0232      | 0.0307      | 0.0608      |
| RMS    | 0.0437      | 0.0282      | 0.0369      | 0.0638      |

issued to the `reference_node`, which streams the spline reference. Deployed control gains are listed in Table VII.

### D. Traceability and Tests

Satisfies FR-2, FR-6, FR-8. Figure 6 shows the UAV tracking the commanded boustrophedon survey trajectory under OptiTrack feedback, both as an *xy* path and as per-axis reference-versus-measured time series; the resulting absolute tracking error is summarized in Fig. 7 and quantified in Table VIII. Across the full survey the mean absolute error stays at or below 3.8 cm per axis ( $\text{RMS} \leq 4.4$  cm), with worst-case excursions of  $\sim 14$  cm at the trajectory turning points, well within the overlap margin required for stitching.

## VII. AERIAL MAPPING AND IMAGE STITCHING

### A. Role and Requirements

`arena_map_builder` converts downward UAV video into a metric occupancy grid at 5 cm/cell, plus marker po-

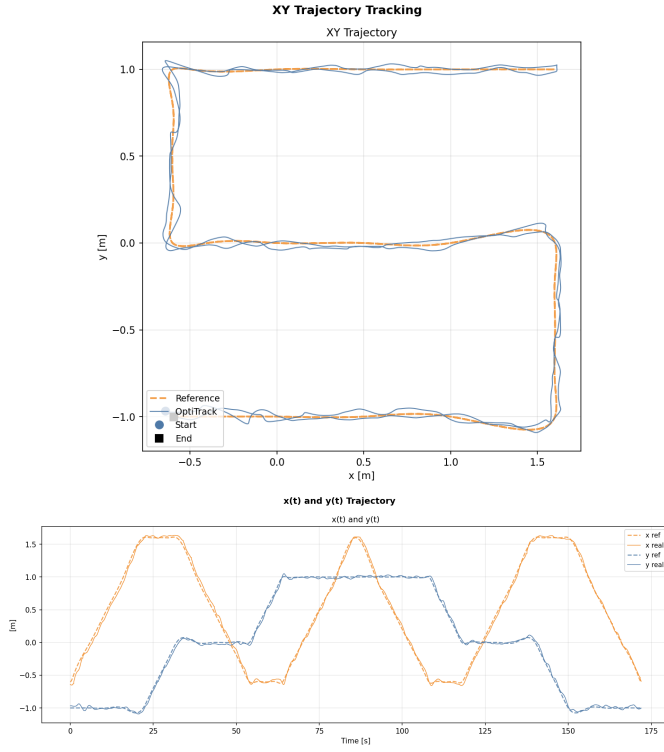


Fig. 6. UAV survey tracking under OptiTrack feedback. Top: commanded vs. measured  $xy$  trajectory. Bottom: per-axis reference  $x(t)$ ,  $y(t)$  against the measured response.

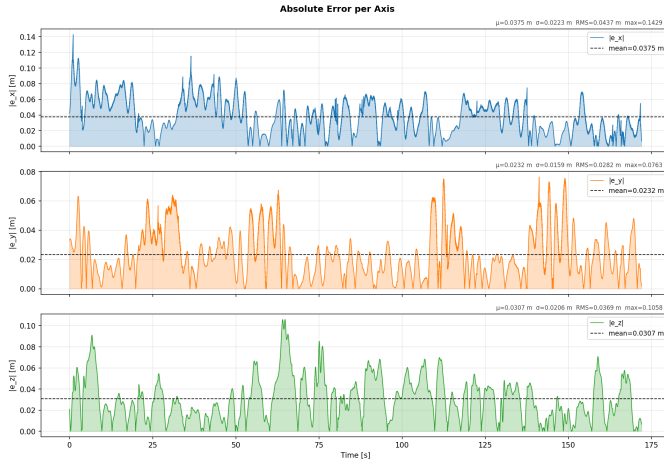


Fig. 7. Absolute UAV tracking error per axis over the survey, with per-axis mean, median, RMS, and peak. Mean error remains a few centimetres per axis.

sitions and a diagnostic feature vector. It satisfies FR-2 and FR-5 and must fit the Jetson memory budget (NFR-5).

## B. Theory and Design Rationale

1) *Similarity transform over full homography*: Top-down flight over a planar arena is a 4-DOF rigid-plus-scale motion. A full 8-DOF homography has extra degrees of freedom that RANSAC will use to fit the repetitive blue-tape grid, producing

compounding “fan” artifacts across the hundreds of frames of a survey. We therefore estimate a similarity transform

$$\mathbf{H}_{\text{sim}} = \begin{pmatrix} a & -b & t_x \\ b & a & t_y \\ 0 & 0 & 1 \end{pmatrix}, \quad a = s \cos \theta, \quad b = s \sin \theta, \quad (5)$$

via `estimateAffinePartial2D` with RANSAC, promoted to  $3 \times 3$  for composition onto the global canvas.

2) *Feature extraction with exclusion masking*: The production front-end can be configured to work with SIFT (5000 keypoints/frame, 128-D descriptors). The arena’s blue adhesive grid is masked out by an HSV threshold (dilated 5 px) before detection: without this mask, RANSAC latches onto visually identical inter-cell tape junctions and converges on the wrong cell. A GPU alternative front-end uses SuperPoint [10] (256-D) with LightGlue matching [11], exported to ONNX and run on the Jetson GPU—the embedded deep-learning component of the system.

3) *Matching pipeline*: Frame pairs are matched by (i)  $k$ -nearest-neighbour ( $k=2$ ) L2 matching, (ii) Lowe’s ratio test at  $r = 0.65$ , (iii) a mutual cross-check requiring the ratio test to pass in both directions, and (iv) a median-absolute-deviation (MAD) displacement pre-filter that removes wrong-cell clusters *before* RANSAC:

$$\boldsymbol{\mu} = \text{median}_i(\boldsymbol{\Delta}_i), \quad \text{MAD} = \text{median}_i \|\boldsymbol{\Delta}_i - \boldsymbol{\mu}\|, \quad (6)$$

keeping match  $i$  iff  $\|\boldsymbol{\Delta}_i - \boldsymbol{\mu}\| < \kappa \cdot \text{MAD}$  with  $\kappa = 4.0$ . Survivors feed a similarity RANSAC (threshold 3 px, 5000 iterations, confidence 0.999).

4) *Canvas and blending*: A fixed  $8000 \times 8000$  canvas is pre-allocated; each frame is warped only over its region of interest ( $\sim 6$  MB vs.  $\sim 140$  MB for the full canvas). Frames are merged with a 4-level Laplacian pyramid blend [6] whose weight  $\alpha(p) = d_{\text{new}} / (d_{\text{new}} + d_{\text{old}})$  comes from a dual distance transform, placing the seam along the iso-depth contour.

5) *Obstacle transfer and consistency scoring*: Obstacles (pink) and walls (green/brown) are transferred from the source frames onto the canvas by two independent projection methods—a bounding-box projection and a per-cell grid projection. Because the two methods are derived differently, their agreement is a proxy for how reliably an obstacle was reconstructed. Three proxies are combined into a per-obstacle confidence  $c \in [0, 1]$ :

- $s_A$  (*centroid agreement*): how closely the obstacle centroids from the two projection methods coincide—small displacement  $\rightarrow s_A \rightarrow 1$ .
- $s_B$  (*area ratio*): the ratio of the smaller to the larger of the two projected obstacle areas—consistent extent  $\rightarrow s_B \rightarrow 1$ .
- $s_C$  (*perturbation stability*): how stable the obstacle is under small perturbations of the projection, penalizing detections that move or disappear under slight changes.

These are merged into the weighted score

$$c = \frac{w_A s_A + w_B s_B + w_C s_C}{w_A + w_B + w_C}, \quad (7)$$



with  $(w_A, w_B, w_C) = (0.45, 0.25, 0.30)$ : centroid agreement is weighted highest as the most direct evidence of a correct reconstruction. Occupancy is then rasterized with confidence-weighted halos of width  $t(c) = t_{\text{base}}e^{-3c}$ , interpolating from free to occupied around each obstacle core, so that low-confidence obstacles are surrounded by a wider cautionary margin.

6) *Learned map-quality gate*: A RandomForest classifier [9] consumes a 46-feature diagnostic vector and outputs a binary “map acceptable” decision  $P(\text{pass} \mid \mathbf{f}) \geq \tau$ . Inference is pure NumPy (a serialized `forest.npz`, no scikit-learn at runtime); missing features map to a  $-1$  sentinel for safe-fail. A rejected map triggers the frontier-exploration fallback of the mission FSM. The training and explainability of this classifier are detailed in Sec. VII-D.

### C. Implementation

The builder runs online (incrementally during flight, hiding stitching latency behind flight time) or offline (post-landing on saved video). Memory is bounded (NFR-5) by a streaming frame generator, the pre-allocated canvas, ROI warping, a keyframe cap of 20, and periodic `malloc_trim`; peak working set is  $\sim 287$  MB. A 335-configuration parameter sweep fixed the deployed baseline (SIFT,  $r=0.65$ ,  $\kappa=4.0$ , lookback 8, keyframe interval 10, processing scale 0.5, blue-tape exclusion).

### D. Traceability and Tests

Satisfies FR-2, FR-5, FR-11, NFR-5. The pipeline reconstructs a full-coverage metric map of the arena that the AMR uses for planning, within the Jetson memory budget (NFR-5), and the learned classifier gates unusable maps to the frontier-exploration fallback (FR-11). Figure 8 shows the feature-exclusion mask derived from the blue-tape grid, the resulting masked stitch, the final occupancy product, and the obstacle transfer projected onto the grid.

1) *Map-quality classifier training and explainability*: The 46 diagnostic features (Groups 1–6, prefixes `stitch_`, `dewarp_`, `green_`, `blob_`, `marker_`, `stitcher_`, `pg_`) are extracted by `extract_features.py` from each `BuildArenaMap` run; the configuration columns (`cfg_*`) are deliberately excluded because they are unavailable at inference time, when the model sees only live pipeline output. Runs flagged `diag_error` or labelled `unsure/unlabeled` are dropped. A 20% stratified hold-out is reserved for evaluation, and a Random Forest (and an XGBoost baseline) is tuned by 50-iteration randomized search over a 5-fold stratified cross-validation, optimizing average precision (area under the precision–recall curve). Class imbalance is handled with `class_weight='balanced_subsample'`, which recomputes weights on each bootstrap sample.

The operating threshold is *not* the default 0.5. Because a false positive (accepting a bad map) is more costly than a false negative (triggering frontier exploration unnecessarily), the deployed threshold is the lowest value at which the precision of the `pass` class reaches a target precision on the

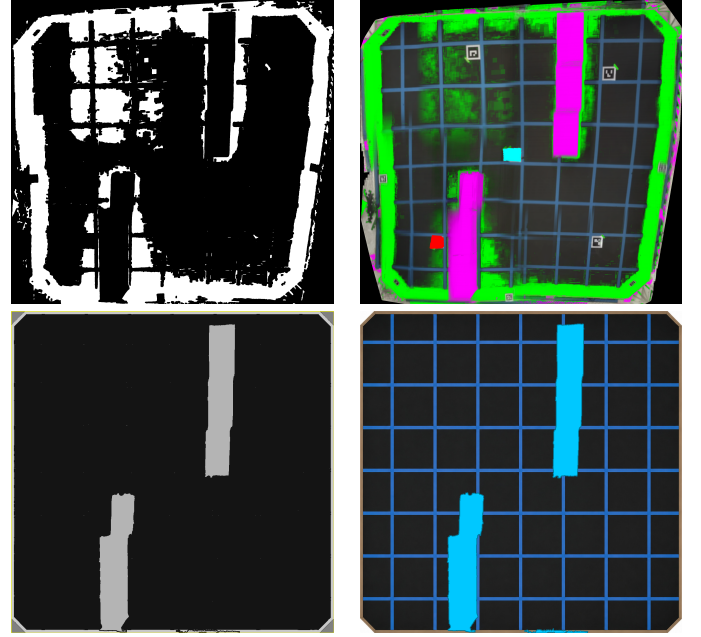


Fig. 8. Stitching pipeline products. Top: the blue-tape exclusion mask (left) and the masked incremental stitch (right). Bottom: the final stitched arena map (left) and the obstacle transfer projected onto the grid (right). The mask prevents RANSAC from matching visually identical inter-cell tape junctions; the transfer projection feeds the centroid- and area-agreement consistency proxies.

hold-out set; it is stored in `threshold.json` and applied at inference. Feature importance is examined through both impurity-based importance and SHAP values (Fig. 9), which confirm the model keys on sensible diagnostics—stitching residuals, green-mask wall coverage, and blob statistics — rather than spurious correlations.

## VIII. AERIAL MARKER LOCALIZATION

### A. Role and Requirements

`arena_marker_localizer` estimates the world-frame pose of each ArUco goal marker from the UAV’s aerial video and synchronized OptiTrack log, satisfying FR-3 (Fig. 10). Position is taken from the stitcher (whose metric pixel projection is more accurate) and *orientation* from PnP, which depends only on relative image geometry and is unaffected by map-origin shifts.

### B. Theory and Design Rationale

1) *Mirror rig and camera transform*: A  $45^\circ$  mirror redirects the forward camera to nadir. The calibrated camera-to-drone transform applies a roll of  $\pi$  (mapping the forward optical axis to downward) and a yaw of  $0.0456$  rad ( $2.6^\circ$  mounting misalignment), with a small translational lever arm.

2) *Planar PnP*: Each detected marker yields four corner correspondences solved by `IPPE_SQUARE`, a closed-form planar method that reduces the 3D–2D problem to a homography and recovers two candidate poses (the planar ambiguity), selecting the one with lower reprojection error. Detections with reprojection error above 4 px are discarded.

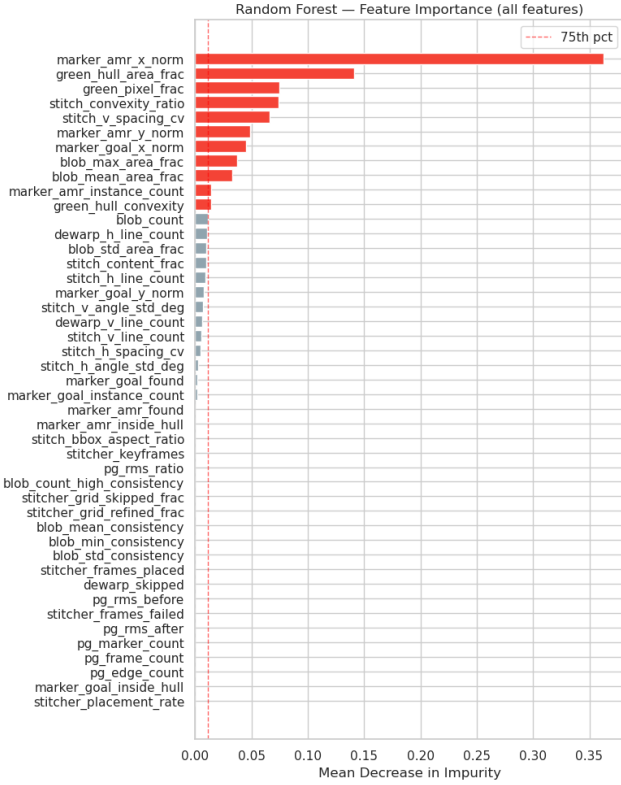


Fig. 9. Random-forest impurity-based feature importance over the 46 diagnostic features. The highest-ranked features correspond to stitching-residual and wall/blob diagnostics, indicating the classifier has learned physically meaningful predictors of map quality.

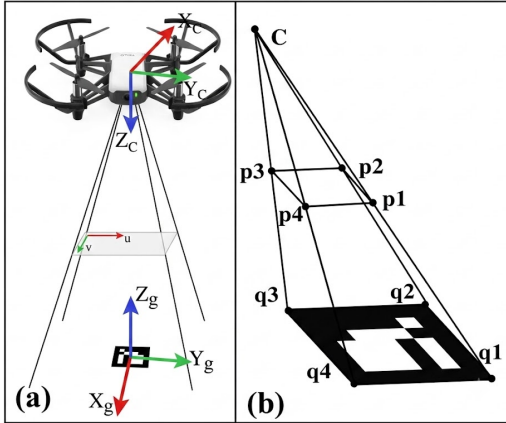


Fig. 10. The UAV detecting ArUco markers in its nadir (mirror-redirected) view during the survey flight.

3) *Transform chain*: A marker pose is composed along a four-link chain,

$$\mathbf{T}_{\text{map} \leftarrow \text{marker}} = \mathbf{T}_{\text{map} \leftarrow \text{opti}} \mathbf{T}_{\text{opti} \leftarrow \text{drone}}(t) \times \mathbf{T}_{\text{drone} \leftarrow \text{cam}} \mathbf{T}_{\text{cam} \leftarrow \text{marker}}(t), \quad (8)$$

with all links as  $4 \times 4$  homogeneous transforms and the OptiTrack log aligned row-by-row to video frames (Fig. 11).

TABLE IX  
LOCALIZEMARKERS SERVICE INTERFACE

|          | Field         | Type         |
|----------|---------------|--------------|
| Request  | video_path    | string       |
|          | optitrack_csv | string       |
| Response | success       | bool         |
|          | message       | string       |
|          | markers       | MarkerPose[] |

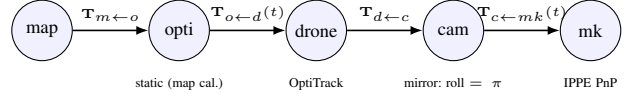


Fig. 11. Four-link transform chain composing each marker pose into the map frame,  $\mathbf{T}_{\text{map} \leftarrow \text{marker}} = \mathbf{T}_{\text{map} \leftarrow o} \mathbf{T}_{o \leftarrow d}(t) \mathbf{T}_{d \leftarrow c} \mathbf{T}_{c \leftarrow mk}(t)$ . The time-varying links come from OptiTrack and from PnP; the camera-to-drone link encodes the  $45^\circ$  mirror (roll =  $\pi$ ).

4) *Robust aggregation*: Each marker accrues 120–180 observations. These pass a per-axis and circular MAD gate ( $k = 3.5$ ), after which position is aggregated by the Weiszfeld geometric median [13] (breakdown point  $\sim 50\%$ ),  $\mathbf{x}^{(k+1)} = \sum_i w_i \mathbf{p}_i / \sum_i w_i$  with  $w_i = 1 / \|\mathbf{p}_i - \mathbf{x}^{(k)}\|$ , and yaw by the circular median  $\hat{\theta} = \text{atan2}(\text{median} \sin \theta_i, \text{median} \cos \theta_i)$ .

5) *Bias calibration*: A four-term linear error model—map-origin shift, camera lever arm, radial scale bias, and yaw misalignment—is fit by iteratively reweighted least squares with Huber weights and physical bounds, validated by leave-one-scan-out cross-validation. An SVD condition-number check detects attitude degeneracy and falls back to a reduced model.

### C. Implementation

The subsystem exposes a `LocalizeMarkers` service (`/localize_markers`) invoked in mission Stage 04 as the orientation source, joined with the map-builder position to publish the world-frame goal pose on `/aruco/goal/pose` (FR-3) and the AMR fix on `/aruco/amr/pose` that the alignment node uses to lock odom to world (FR-9). The request carries the absolute paths to the drone video and the per-frame OptiTrack CSV; the response returns a success flag, a status message, and an array of `MarkerPose` entries—one per accepted marker, with rejected markers absent (Table IX). Each `MarkerPose` reports the marker ID, full 3D pose and floor-plane 2D pose in the map frame, a `PoseWithCovarianceStamped` (diagonal covariance  $\sigma_{xx}^2 = \sigma_{yy}^2 = \text{base\_var}_{\text{pos}}/n_{\text{obs}}$ ,  $\sigma_{\psi}^2 = \text{base\_var}_{\text{yaw}}/n_{\text{obs}}$ ), the occupancy-grid cell indices, and the number of contributing observations  $n_{\text{obs}}$ .

The bias-calibration model is validated by leave-one-scan-out cross-validation (Sec. VIII); the four-link transform chain it corrects is shown in Fig. 11.



## IX. AMR STATE ESTIMATION: EXTENDED KALMAN FILTER

### A. Role and Requirements

`ekf_amr` is the AMR’s state estimator and the single source of state for every downstream module. It fuses wheel-encoder velocity, IMU gyroscope, visual-odometry (VIO) forward velocity, and absolute ArUco wall-marker fixes (§IX-B6) into a continuous pose on `/amr/ekf/odom` and the `odom`  $\rightarrow$  `base_footprint` transform. It is implemented from scratch (NFR-1). The GPS-denied design (no positioning markers *on* the robot and no external motion capture) means absolute correction comes only from onboard observation of surveyed environmental fiducials; the noisy, loop-closure-spiked VIO stream demands adaptive gating that stock packages do not provide.

### B. Theory and Design Rationale

1) *State and process model*: The state  $\mathbf{X} = [x, y, \theta, v, \omega]^\top$  carries velocity as explicit states so heterogeneous velocity sources—and future absolute-position fixes—all enter through standard Kalman updates. Between steps it evolves under a unicycle model with a constant-velocity assumption,

$$f(\mathbf{X}) = [x + T_s v \cos \theta, y + T_s v \sin \theta, \theta + T_s \omega, v, \omega]^\top, \quad (9)$$

with Jacobian

$$F = \begin{pmatrix} 1 & 0 & -T_s v \sin \theta & T_s \cos \theta & 0 \\ 0 & 1 & T_s v \cos \theta & T_s \sin \theta & 0 \\ 0 & 0 & 1 & 0 & T_s \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}, \quad (10)$$

and process noise  $Q = \text{diag}(10^{-4}, 10^{-4}, 10^{-4}, 0.05, 0.05)$ : pose is trusted to propagate, while loose velocity noise lets measurements correct  $(v, \omega)$  freely. The prediction is  $\hat{\mathbf{X}}^- = f(\mathbf{X}_{k-1})$ ,  $P^- = F P F^\top + Q$ , with heading wrapped to  $(-\pi, \pi]$ .

2) *Measurement updates*: Three updates fire sequentially each step (wheel, IMU, VIO), skipping sensors with no new reading. Wheel odometry observes  $(v, \omega)$  with  $R_{\text{wheel}} = \text{diag}(0.2, 0.2)$  (slip and quantization). The IMU observes  $\omega$  after subtracting a calibrated bias  $b = 0.002618 \text{ rad s}^{-1}$ , with a tight  $R_{\text{imu}} = \text{diag}(0.001)$ . VIO contributes only its forward velocity  $v_{x,\text{vio}}$  ( $R_{\text{vio}} = \text{diag}(0.1)$ ); its position is discarded because VSLAM position drifts and jumps at loop closures.

3) *VIO source and the loop-closure spike*: The VIO stream is produced by *cuVSLAM* [18], the CUDA-accelerated stereo visual-inertial SLAM engine shipped as Isaac ROS Visual SLAM [19] and run on the Jetson Orin Nano from the RealSense D435i. Its *front end* tracks sparse image features across the stereo pair and, between keyframes, estimates incremental ego-motion by minimizing reprojection (and IMU pre-integration) residuals over a sliding window—this is the low-latency visual odometry that drives `twist.twist.linear.x`. Its *back end* maintains a pose graph and, using place recognition, detects when the camera

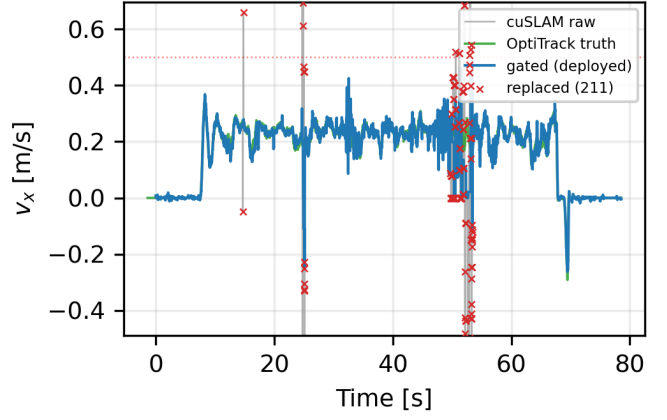


Fig. 12. Raw *cuVSLAM* forward velocity, OptiTrack truth, and the deployed adaptive gate output. The gate replaces the loop-closure impulse samples (red) with the last good estimate, tracking the true  $\sim 0.2 \text{ m s}^{-1}$  cruise.

revisits an earlier location; on such a *loop closure* it inserts a loop constraint and runs a global pose-graph optimization that redistributes the accumulated drift over the whole trajectory.

This back-end correction is what corrupts the velocity. Because the published linear velocity is effectively the time derivative of the optimized pose track, a loop closure’s near-instantaneous repositioning of the current keyframe appears as a one- or two-sample *impulse* in  $v_{x,\text{vio}}$ , even though the robot moved smoothly—the displacement is an artefact of the estimator re-anchoring, not of physical motion. The same correction injects metre-scale discontinuities into the position estimate, which is why only the velocity scalar is fused and why even that scalar must be gated: on our runs these impulses reach  $|v_x| \approx 4.9 \text{ m s}^{-1}$ , more than  $20\times$  the robot’s true  $\sim 0.2 \text{ m s}^{-1}$  cruise (Fig. 12), and they arrive in short bursts as a closure event settles.

4) *Adaptive VIO gate*: A fixed-cutoff Butterworth low-pass filter could not suppress loop-closure spikes—its transient response to a sharp spike propagated into the filter and caused sustained position deviations (Fig. 13). The deployed `ema_gate` is a two-stage adaptive filter. Stage P0 holds the last good value when  $|v_x| > 0.5 \text{ m s}^{-1}$  (the robot’s mechanical limit). Stage P1 tracks online EMA estimates of mean and variance,

$$\hat{\mu}_k = \alpha v_k + (1 - \alpha) \hat{\mu}_{k-1}, \quad \hat{v}_k^2 = \alpha v_k^2 + (1 - \alpha) \hat{v}_{k-1}^2, \quad (11)$$

$\hat{\sigma}^2 = \max(0, \hat{v}_k^2 - \hat{\mu}_k^2)$ , and gates against  $\tau_k = \max(2.0 \hat{\sigma}_k, 0.17)$ , the floor preventing collapse near stand-still. Because spikes arrive in clusters, a 10-sample window of spike decisions triggers *burst mode* above a 0.30 spike fraction, during which all samples are rejected and the statistics update slowly ( $\alpha_{\text{burst}} = 0.02$ ). A 20-sample warmup seeds the statistics.

5) *Covariance update*: All updates use the Joseph form  $P^+ = (I - KH)P^-(I - KH)^\top + KRK^\top$ , preserving symmetry and positive semi-definiteness to machine precision.

**Algorithm 1** P1 adaptive EMA gate (per VIO sample)

---

```

 $in\_burst \leftarrow \text{mean}(\text{recent\_window}) \geq 0.30$ 
 $\sigma^2 \leftarrow \max(0, \text{ema\_sq} - \text{ema}^2)$ 
 $\tau \leftarrow \max(2.0\sqrt{\sigma^2}, 0.17)$ 
 $\text{spike} \leftarrow |v_x - \text{ema}| > \tau$ 
if  $in\_burst$  or  $\text{spike}$  then
     $v_{out} \leftarrow \text{last\_good}$ 
    update  $\text{ema}, \text{ema\_sq}$  with  $\alpha_{burst}$ ,  $\text{clip}(v_x, -0.5, 0.5)$ 
else
     $\text{last\_good} \leftarrow v_x$ ;  $v_{out} \leftarrow v_x$ 
    update  $\text{ema}, \text{ema\_sq}$  with  $\alpha$ ,  $v_x$ 
end if
append  $\text{spike}$  to  $\text{recent\_window}$ ; return  $v_{out}$ 

```

---

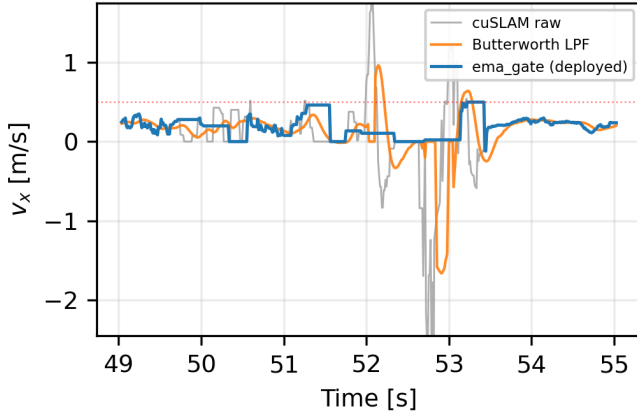


Fig. 13. Filter ablation over a loop-closure burst: the Butterworth low-pass filter rings and overshoots in response to the spikes, whereas the deployed `ema_gate` holds a flat estimate close to the true velocity.

6) *Absolute anchoring from ArUco wall markers*: The three velocity sources above are proprioceptive: none observes absolute position, so on their own the filter only dead-reckons and  $(x, y)$  drift grows unbounded. The absolute correction is supplied by `aruco_localizer`, a fourth, asynchronous measurement source that observes the four ArUco markers mounted at known poses on the arena walls (the GPS-denied design forbids markers *on* the robot or external mocap, but environmental fiducials at surveyed locations are exactly the “visual landmarks” the problem statement permits). It runs continuously from mission Stage 08 onward and counteracts drift whenever a marker is in view.

For each detected marker the node solves an IPPE\_SQUARE PnP from the four corners to obtain the camera $\leftarrow$ marker transform, then composes the robot pose in the world frame from the surveyed marker pose and the cached camera extrinsic,

$$\mathbf{T}_{bf}^{\text{world}} = \mathbf{T}_{ar}^{\text{world}} (\mathbf{T}_{ar}^{\text{cam}})^{-1} (\mathbf{T}_{cam}^{\text{bf}})^{-1}. \quad (12)$$

When several markers are visible, their estimates are fused by inverse-square-distance weighting,  $w_k = 1/(d_k^2 + \varepsilon)$  with  $d_k = \|\mathbf{t}_k\|$ , taking a weighted mean of the translations and

TABLE X  
EKF DEPLOYED PARAMETERS (SELECTED)

| Parameter                      | Value                         | Meaning              |
|--------------------------------|-------------------------------|----------------------|
| <code>sampling_time</code>     | 0.01 s                        | step period (100 Hz) |
| <code>Q diag</code>            | $10^{-4}$ (pose), 0.05 (vel.) | process noise        |
| <code>R<sub>wheel</sub></code> | [0.2, 0.2]                    | wheel $[v, \omega]$  |
| <code>R<sub>imu</sub></code>   | [0.001]                       | gyro $[\omega]$      |
| <code>R<sub>vio</sub></code>   | [0.1]                         | VIO $[v_x]$          |
| <code>p0_vx_max</code>         | $0.5 \text{ m s}^{-1}$        | physical limit       |
| <code>pl_n_sigma</code>        | 2.0                           | gate width           |
| <code>pl_burst_thresh</code>   | 0.30                          | burst entry fraction |

a hemisphere-aligned weighted average of the quaternions—closer markers, whose angular resolution is higher, dominate. The reported uncertainty follows a distance-linear model

$$\sigma_{xy} = \sigma_{xy,0} + \alpha_{xy} d, \quad \sigma_{\theta} = \sigma_{\theta,0} + \alpha_{\theta} d, \quad (13)$$

published as a `PoseWithCovarianceStamped` on `/aruco_pose` with the  $z$ , roll, and pitch entries masked out (set to a large variance) so the EKF corrects only the in-plane states. The filter consumes it through a linear absolute update  $H_{\text{aruco}} = [I_{3 \times 3} | 0_{3 \times 2}]$  on  $(x, y, \theta)$ , requiring no structural change. Detections beyond 2.5 m are rejected, and during gaps—when all corners are occluded or out of range—the filter reverts to proprioceptive dead-reckoning until the next sighting.

### C. Implementation

The EKF step runs on a fixed 100 Hz timer independent of sensor arrival; callbacks buffer readings behind `_new` flags, and a step with no new data returns early to avoid spurious drift. Parameters appear in Table X.

### D. Traceability and Tests

Satisfies FR-1, FR-10, NFR-1, NFR-7. The filter is evaluated against OptiTrack ground truth over a representative navigation run, with wheel odometry alone as a baseline (Figs. 14–15). The EKF estimate stays tightly locked to ground truth while pure wheel odometry drifts away, especially after sustained rotation. Quantitatively, the EKF holds a mean absolute position error of 4.6 cm in  $x$  and 6.3 cm in  $y$  (RMS 7.3 cm and 8.5 cm), against 52.3 cm and 41.6 cm for wheel odometry—an  $\sim 88\%$  reduction in mean position error from fusing IMU and gated visual odometry. The VIO gate itself is characterized in Figs. 12 and 13.

## X. AMR DRIVE STACK: ODOMETRY AND CONTROL

### A. Role and Requirements

`amr_bringup` is the RDK X3 hardware layer: `driver_node` (hardware abstraction), `wheel_odometry_node` (dead-reckoning), and `controller_node` (reference tracking). Together they close a sensing–estimation–control loop at the drive level, decoupled from planning.

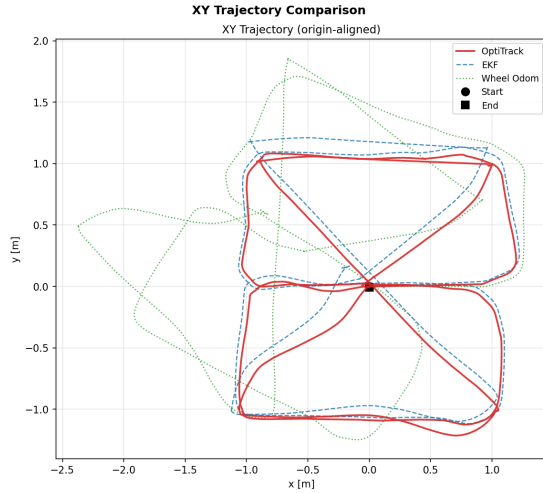


Fig. 14. Origin-aligned  $xy$  trajectory of the AMR: OptiTrack ground truth, the EKF estimate, and wheel odometry. The EKF tracks ground truth closely while wheel odometry accumulates drift.



Fig. 15. Top: per-axis position  $x(t)$ ,  $y(t)$  for ground truth, EKF, and wheel odometry. Bottom: absolute position error versus OptiTrack for the EKF and for wheel odometry, with per-axis mean and RMS.

## B. Theory and Design Rationale

1) *Hardware abstraction*: `driver_node` wraps the SunriseRobot SDK and is the only node touching hardware. It forwards `/amr/cmd_vel` to `set_car_motion`, reads wheel velocity, IMU, and battery at 100 Hz, and enforces emergency stop: a `False`→`True` transition immediately commands zero

motion and a three-beep alert; the clear transition resumes commands. The IMU is mounted non-canonically, so a fixed axis permutation and sign flip maps the SDK frame to the ROS body frame,

$$(a_x, a_y, a_z)_{\text{ROS}} = (-a_x^s, a_z^s, a_y^s), \quad (14)$$

and identically for the gyroscope; the magnetometer is unused.

2) *Dead-reckoning odometry*: Though mechanically a mecatum platform, the base is treated as a differential-drive robot: lateral velocity is discarded ( $\dot{y}_{\text{body}} = 0$  by construction), since planning never commands lateral motion. The pose integrates by Euler,

$$x_{k+1} = x_k + v_x \cos \theta_k \Delta t, \quad y_{k+1} = y_k + v_x \sin \theta_k \Delta t, \quad (15)$$

$\theta_{k+1} = \text{wrap}(\theta_k + \omega_z \Delta t)$ , with a  $\Delta t$  guard ( $0 < \Delta t \leq 1$  s) and unit calibration scales (the SDK velocities need no correction at the arena scale).

3) *Feedback-linearizing controller*: The planner publishes a world-frame reference  $(x_r, y_r, \theta_r)$  with velocity  $(\dot{x}_r, \dot{y}_r)$ . Controlling the centre directly is singular for the unicycle, so the controller acts on a virtual point  $R = 0.2$  m ahead,  $x_R = x + R \cos \theta$ ,  $y_R = y + R \sin \theta$ . Virtual proportional-plus-feedforward inputs  $u_1 = k_{px}(x_r - x_R) + \dot{x}_r$ ,  $u_2 = k_{py}(y_r - y_R) + \dot{y}_r$  ( $k_{px} = 1.0$ ,  $k_{py} = 2.0$ ) are mapped to body commands by inverting

$$\begin{pmatrix} \dot{x}_R \\ \dot{y}_R \end{pmatrix} = \underbrace{\begin{pmatrix} \cos \theta & -R \sin \theta \\ \sin \theta & R \cos \theta \end{pmatrix}}_{T(\theta)} \begin{pmatrix} v \\ \omega \end{pmatrix}, \quad (16)$$

which is invertible everywhere ( $\det T = R \neq 0$ ), giving  $v = u_1 \cos \theta + u_2 \sin \theta$  and  $\omega = (-u_1 \sin \theta + u_2 \cos \theta)/R$  and the decoupled error dynamics  $\dot{e}_x = -k_{px}e_x$ ,  $\dot{e}_y = -k_{py}e_y$ . Drivetrain stiction is compensated by a deadband offset,  $v_{\text{cmd}} = \text{sgn}(v)(|v| + 0.15)$  for  $|v| > 0.001 \text{ m s}^{-1}$ .

## C. Implementation

The controller reads pose from the world  $\rightarrow$  `base_footprint` transform rather than a fixed odometry topic, making it localization-agnostic; all three nodes run 100 Hz loops.

## D. Traceability and Tests

Satisfies FR-1, FR-4 (emergency-stop actuation), NFR-7. Figure 16 shows the AMR following a planned reference around the obstacles on the occupancy map: the feedback-linearizing controller keeps the executed path close to the planned trajectory and the OptiTrack ground truth. Aggregate tracking error over full navigation runs is reported in Sec. XVII.

## XI. LiDAR SENSING AND OCCUPANCY MAPPING

### A. Role and Requirements

`oradar_ros` drives the LiDAR and `world_mapper` maintains the AMR's running occupancy map, jointly satisfying FR-4 and NFR-2.

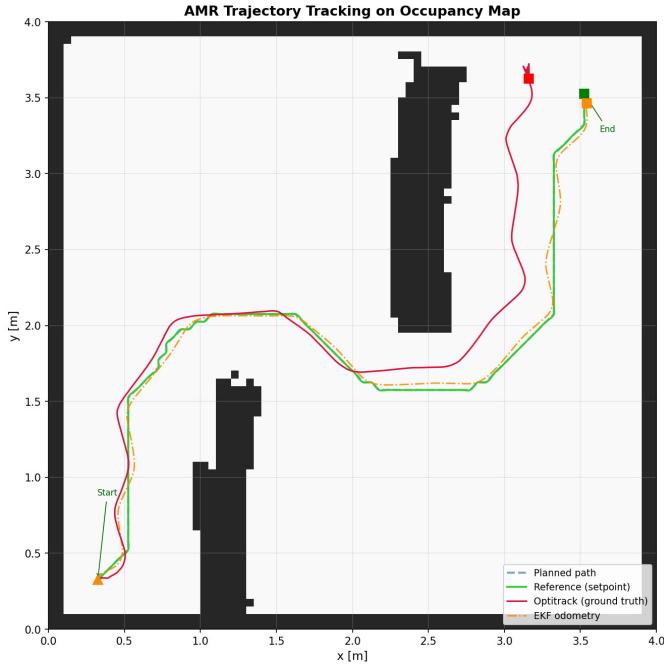


Fig. 16. AMR tracking a planned reference trajectory on the occupancy map under the feedback-linearizing controller: planned path, reference setpoint, OptiTrack ground truth, and EKF estimate.

## B. Theory and Design Rationale

1) *LiDAR driver*: The OraDAR MS200 streams OradarLidarFrame packets (12 points each, CRC-8 checked, distances in mm, angles in centi-degrees). The driver converts clockwise to counter-clockwise ( $angle \rightarrow 360 - angle$ ), gates ranges to 0.05 m to 20 m, maps returns onto a regular angular grid (nearest return wins on collision), and fills single-cell gaps by interpolation. A static transform places the sensor at  $x = 0.07$  m,  $z = 0.13$  m on `base_footprint`; the scan timestamp is the acquisition start, bounding positional uncertainty to  $\leq 10$  mm at operating speed.

2) *Bayesian occupancy update*: A natural first instinct is to store  $P(m_{ij} = 1)$  directly and update it with Bayes' rule after each observation  $z$ :

$$P(m | z_{1:t}) = \frac{P(z_t | m) P(m | z_{1:t-1})}{P(z_t | z_{1:t-1})}$$

This formulation has two critical numerical problems:

- 1) **Probability underflow**. After many “free” observations,  $P(m = 1)$  approaches 0. Multiplying many small numbers leads to floating-point underflow ( $0.1^{100} \approx 10^{-100}$ ), resulting in exact zeros that can never recover from a single occupied observation.
- 2) **Cell freezing**. Conversely, a cell that reaches  $P = 1$  can never be set to free again because  $\eta \cdot P(z_{\text{free}} | m = 1) \cdot 1 = 0$  (there is no numerical room left).

The log-odds (or *logit*) transform solves both problems by mapping  $[0, 1]$  to  $(-\infty, +\infty)$ :

$$\ell(m) = \log \frac{P(m = 1)}{P(m = 0)} = \log \frac{P(m = 1)}{1 - P(m = 1)} \quad (17)$$

Under this representation, the recursive Bayesian update becomes a simple **addition**:

$$\ell_t(m) = \ell_{t-1}(m) + \underbrace{\log \frac{P(z_t | m = 1)}{P(z_t | m = 0)}}_{\text{log-odds of sensor model}} - \underbrace{\log \frac{P(m = 1)}{P(m = 0)}}_{\text{log-odds prior}} \quad (18)$$

**Derivation sketch.** Starting from the binary Bayes filter, applying the log-odds transform to both sides of the posterior, and using the Markov assumption (each scan is conditionally independent), the recursive product of likelihoods becomes a sum:

$$\ell_t = \ell_{t-1} + \ell_{\text{sensor}}(z_t)$$

This is numerically stable over thousands of updates because we are adding and subtracting bounded increments, not multiplying probabilities.

In practice, the full inverse-sensor-model integral is approximated by two scalar constants:

| Parameter                     | Symbol               | Typical value |
|-------------------------------|----------------------|---------------|
| Log-odds increment — occupied | $\ell_{\text{occ}}$  | +0.85         |
| Log-odds increment — free     | $\ell_{\text{free}}$ | -0.40         |
| Prior (unknown)               | $\ell_0$             | 0.0           |
| Clamp minimum                 | $\ell_{\text{min}}$  | -5.0          |
| Clamp maximum                 | $\ell_{\text{max}}$  | +5.0          |

TABLE XI

LOG-ODDS CONSTANTS USED BY NODEWORLD\_MAPPER. THE ASYMMETRY ( $|\ell_{\text{occ}}| > |\ell_{\text{free}}|$ ) MEANS IT TAKES MORE EVIDENCE TO MARK A CELL OCCUPIED THAN TO CLEAR IT, WHICH REDUCES FALSE POSITIVES FROM NOISY SCANS.

Clamping at  $[\ell_{\text{min}}, \ell_{\text{max}}]$  bounds how many consecutive observations are needed to change a cell's state in either direction, preventing hysteresis and keeping cells responsive to environmental changes.

3) *Handling of dynamic obstacles*: The arena contains only static obstacles, so no motion model or prediction is implemented. A slowly moving obstacle is nonetheless handled—sub-optimally but without any dedicated machinery—by the continuous log-odds update itself: as the obstacle vacates a cell, repeated free returns drive that cell's log-odds back below the occupancy threshold, while its new location accumulates occupied returns. The hit/miss asymmetry and saturation bounds set how quickly the map forgets a departed obstacle; this lag makes the scheme adequate only for motion slow relative to the mapping rate (FR-13).

## C. Implementation

The driver runs at the deployed 15 Hz; `world_mapper` integrates scans on a 50 Hz timer along the `world`  $\rightarrow$  `odom`  $\rightarrow$  `base_footprint`  $\rightarrow$  `lidar` chain. The `world`  $\rightarrow$  `odom` transform is published by the `alignment_node`





Fig. 17. Occupancy map under construction as the AMR drives: log-odds cells accumulate occupied (dark) and free (light) evidence from successive LiDAR scans.

from the drone-observed AMR fix on `/aruco/amr/pose` (Stage 09.b); when that pose is unavailable it arrives as all-NaN, which the alignment node reads as “no data” and replaces with the trivial identity transform.

#### D. Traceability and Tests

Satisfies FR-4, FR-9, FR-13, NFR-2. Figure 17 shows the occupancy map under construction during a run: the log-odds map converges to the obstacle layout as the AMR drives, updating well above the 1 Hz floor (NFR-2) and reflecting slow obstacle motion through continued free/occupied updates (FR-13).

### XII. MAP FUSION

#### A. Role and Requirements

`fusion` merges the aerial prior with the AMR’s live LiDAR map into the costmap the planner consumes (FR-4, FR-5).

#### B. Theory and Design Rationale

Both maps are published in the same world frame at identical resolution and origin by construction, so *no registration is needed*. Fusion is a single vectorized elementwise pass with a four-case priority rule (drone threshold 65, default fill 25):

$$G_F = \begin{cases} 100 & G_D \geq 65 \\ G_A & G_D = -1 \\ \max(25, G_A) & G_D < 65, G_D \neq -1, G_A \neq -1 \\ 25 & \text{otherwise.} \end{cases} \quad (19)$$

The cases correspond, respectively, to a confirmed drone obstacle (immutable), a drone-unknown cell (defer to the AMR map  $G_A$ ), a drone-free cell that the AMR may raise, and the default fill. Drone free space is treated as 25 rather than 0 because aerial stitching is not certain enough to assert free; the AMR map can only *raise* occupancy, never clear a drone obstacle. An earlier ICP-based registration was abandoned: early in a mission the AMR map is too sparse for stable coarse-search peaks, and exact pre-alignment made registration unnecessary.

#### C. Implementation

The drone map uses `TRANSIENT_LOCAL` QoS so the node receives it even if it starts late (NFR-9); the fused map is republished reactively on each AMR update. The fusion thus satisfies FR-4 and FR-5, supplying the planner with a single costmap that combines the aerial prior and the live LiDAR map.

### XIII. PATH PLANNING AND TRAJECTORY EXECUTION

#### A. Role and Requirements

`trajectory_planner` plans a collision-free path to the goal and emits a trackable reference for the drive stack (FR-1, FR-4, NFR-3).

#### B. Theory and Design Rationale

The fused costmap is inflated by an exponential look-up table  $c(d) = 99 e^{-k(d-r_{\text{robot}})}$  ( $k = 3.5$ , saturating to 100 inside the robot radius). A\* searches the 8-connected grid with the octile heuristic

$$h = \max(\Delta x, \Delta y) + (\sqrt{2} - 1) \min(\Delta x, \Delta y), \quad (20)$$

on occupancy-weighted edge costs. Replanning triggers on goal change, path blockage, elapsed time, or a proximity score  $\sigma_{\text{prox}} = \sum_i e^{-d_i/r}$  exceeding a threshold; the blockage and proximity triggers let the planner route around slowly moving obstacles as the occupancy map updates (FR-13). The waypoint path is smoothed by an arc-length -parameterized cubic spline and given a trapezoidal speed profile (ramp distance  $d_{\text{ramp}} = v^2/2a = 0.225$  m), so the commanded speed adapts to the path geometry. The resulting reference is tracked by the same feedback-linearizing controller as Section X.

#### C. Implementation

Inputs are the fused `/drone/map` and the ArUco goal pose; the node publishes the time-parameterized reference to the drive controller, satisfying FR-1 and FR-4 within the NFR-3 replanning budget. End-to-end navigation performance for this stage is reported in Sec. XVII.

### XIV. FRONTIER-EXPLORATION FALLBACK

#### A. Role and Requirements

When the aerial map-quality classifier rejects the drone map (Sec. V, Stage 04.c), the AMR has neither a usable occupancy prior nor a world-frame pose for the goal marker. `frontier_explorer` is the fallback that recovers the mission autonomously (FR-11): it explores the arena on the



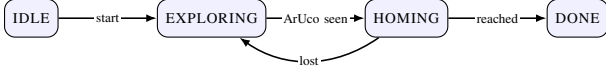


Fig. 18. Frontier-exploration supervisory state machine. Detection loss in HOMING returns control to EXPLORING; a stall watchdog re-issues goals during EXPLORING.

AMR’s own onboard occupancy map, searches for the goal ArUco marker with the RGB camera, and homes onto it once found. It is armed at Stage 11.b and triggered at Stage 12.a by a single Boolean on `/map_fail_fallback/start`; everything thereafter is autonomous.

### B. Theory and Design Rationale

1) *Exploration state machine*: A supervisory state machine arbitrates the mission (Fig. 18). From IDLE the trigger moves it to EXPLORING, where it forwards frontier goals to the existing A\*+spline planner of Sec. XIII; on detecting the goal marker it transitions to HOMING, and to DONE on arrival. A lost detection in HOMING falls back to EXPLORING, and a stall watchdog (no 0.05 m of motion in 4 s) re-issues or abandons the current goal.

2) *Frontier detection and scoring*: A *frontier cell* is a free cell ( $0 \leq v < 90$ ) with at least one unknown neighbour ( $v = -1$ ); frontier cells are grouped by an 8-connected union-find and small clusters are discarded. Each surviving cluster centroid is scored by a normalized blend of proximity and area,

$$\text{score} = w_{\text{dist}} \widehat{\text{prox}} + w_{\text{size}} \widehat{\text{area}}, \quad (21)$$

$(w_{\text{dist}}, w_{\text{size}}) = (0.85, 0.15)$ . Proximity is not raw Euclidean distance but a *heading-inflated* perceived distance: the bearing offset  $\Delta\theta$  between the robot heading and the cluster is penalized by

$$\text{heading\_score} = \left( \frac{1 + \cos \Delta\theta}{2} \right)^3, \quad (22)$$

with clusters more than  $60^\circ$  off-heading multiplied by 0.01. Baking the heading penalty into the effective distance (rather than adding a separate, set-normalized heading term that collapses when all frontiers face the same way) keeps a behind-the-robot frontier perceived as far even when it is geometrically nearest, which suppresses oscillating U-turns. A safety walk pulls each goal back to an obstacle-clear cell, and goals that A\* cannot solve are blacklisted.

3) *Visual-servo homing*: A\* homing targets the *last observed* world pose of the marker, which localization drift and grid resolution can miss by 10–20 cm. The final approach therefore switches to an image-space visual servo that drives purely from the detection, independent of any world-model error. With normalized horizontal pixel error  $e_x = (t_x/t_z)/\tan(\text{hfov}/2)$ ,

$$\omega = -K_\omega e_x, \quad v = K_v (t_z - d_{\text{stop}}) f_{\text{centre}}, \quad (23)$$

where  $f_{\text{centre}}$  gates forward motion until  $|e_x|$  is below a centering threshold, so the robot centres on the marker before advancing and stops at  $t_z \leq d_{\text{stop}}$ .

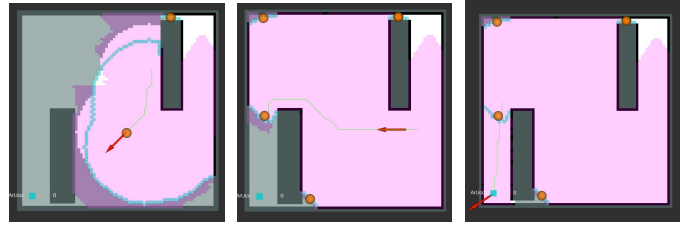


Fig. 19. Frontier-exploration simulation progression (left to right): the robot expands the explored region (pink) into unknown space (grey), tracking frontier cells (cyan) until it reaches and homes onto the goal ArUco marker.

### C. Implementation

The fallback reuses the deployed planner (A\* + spline follower) for both exploration and coarse homing, and the same ArUco detection front-end as the localization stack; only the supervisory controller, frontier scorer, and visual servo are specific to it. It consumes the AMR’s onboard occupancy map and EKF pose and recomputes frontiers at 1 Hz.

### D. Traceability and Tests

Satisfies FR-11. The fallback was validated in simulation: starting from an unknown arena, the robot drives toward the frontier between explored (pink) and unknown (grey) space, progressively reveals the map, and homes onto the goal marker once detected (Fig. 19). The three snapshots show the exploration filling the free space from an early partial sweep to near-complete coverage with the robot converging on the marker.

## XV. PHYSICAL SAFETY: AUTOMATIC EMERGENCY BRAKING

### A. Role and Requirements

`emergency_stop` is an independent, high-rate safety layer decoupled from planning (FR-4, NFR-4).

### B. Theory and Design Rationale

A blind-spot angular mask excludes  $[120^\circ, 240^\circ]$  (chassis structure). An  $N$ -consecutive streak detector commands STOP after  $n = 5$  consecutive scans report an obstacle within 0.3 m, and clears after  $n$  clean scans; at the deployed 15 Hz this is a reaction time of  $\tau \approx 5/15 \approx 333$  ms. Multiple trigger reasons combine in an OR state machine. Safety holds when the stopping distance bound  $d_{\text{min}} > v \cdot (n/f_s) + d_{\text{brake}}$  is satisfied.

### C. Implementation

The node raises `/amr/emergency_stop`, which `driver_node` honors by zeroing motion; the mission FSM health-checks the node at Stage 07.

TABLE XII  
EMERGENCY-BRAKING CHARACTERIZATION

| Metric                  | Mean | Median | Std  | Min  | Max  |
|-------------------------|------|--------|------|------|------|
| Response time (s)       | 0.34 | 0.33   | 0.04 | 0.27 | 0.45 |
| Trigger latency (scans) | 5.1  | 5.0    | 0.6  | 5    | 7    |
| Stopping distance (m)   | 0.18 | 0.17   | 0.03 | 0.13 | 0.25 |
| Clearance at stop (m)   | 0.12 | 0.13   | 0.03 | 0.05 | 0.17 |

| Reliability metric             | Value        | Trials  |
|--------------------------------|--------------|---------|
| True-positive stop rate        | 30/30 (100%) | 30      |
| False triggers (obstacle-free) | 0 in 42 min  | 38 runs |
| Recovery (clear → resume)      | 30/30 (100%) | 30      |

$N = 30$  approach trials at  $v = 0.30 \text{ m s}^{-1}$ , LiDAR at 15 Hz, threshold 0.30 m,  $n_{\text{trig}} = 5$ .

#### D. Traceability and Tests

Satisfies FR-4, FR-12, NFR-4. The braking layer was characterized over repeated approach trials in which the AMR was driven toward a static obstacle at a fixed commanded speed and the latency from threshold breach to a zeroed velocity command was measured, alongside the stopping distance and the false-trigger count during obstacle-free traversals. Table XII summarizes the results, which follow the theoretical bound  $\tau \approx n_{\text{trig}}/f_s \approx 333 \text{ ms}$  (NFR-4): the measured mean response time of 0.34 s matches the 5/15 Hz streak-detector prediction, and no false positives were observed in obstacle-free runs.

### XVI. NETWORK SECURITY MIDDLEWARE

#### A. Role and Requirements

`ros2_security` authenticates and optionally encrypts ROS 2 traffic without DDS-Security or SROS2, satisfying FR-7. It is opt-in per node, with a global kill switch and mixed-trust support.

#### B. Theory and Design Rationale

A self-signed CA (RSA-4096) issues per-node RSA-2048 X.509 certificates whose common name equals the node name; all nodes load the CA certificate to verify peers. Three levels are offered: NONE (pass-through), SIGN (RSA-PSS over the CDR payload), and SIGN\_ENCRYPT (AES-256-GCM then RSA-PSS over the ciphertext). At publish, the payload is serialized, optionally encrypted with a random 12-byte nonce, signed, and wrapped in a `SecureEnvelope` (sender id, timestamp, signature, payload, nonce). At subscribe, the signature is verified *before* any decryption (no decrypt oracle), the sender is checked against the trusted set, the `min_level` policy is enforced, the timestamp is checked against a replay window, and only then is the payload decrypted and deserialized. The AES key is derived deterministically as SHA-256(public-key DER)—the only key material both parties share—so no secret distribution is required. A `LegacyRelayNode` bridges secured and native topologies, and the `ROS2_SECURITY_DISABLED` kill switch degrades the whole system to native ROS 2 uniformly (a mixed state would cause a DDS type mismatch). The complete publish

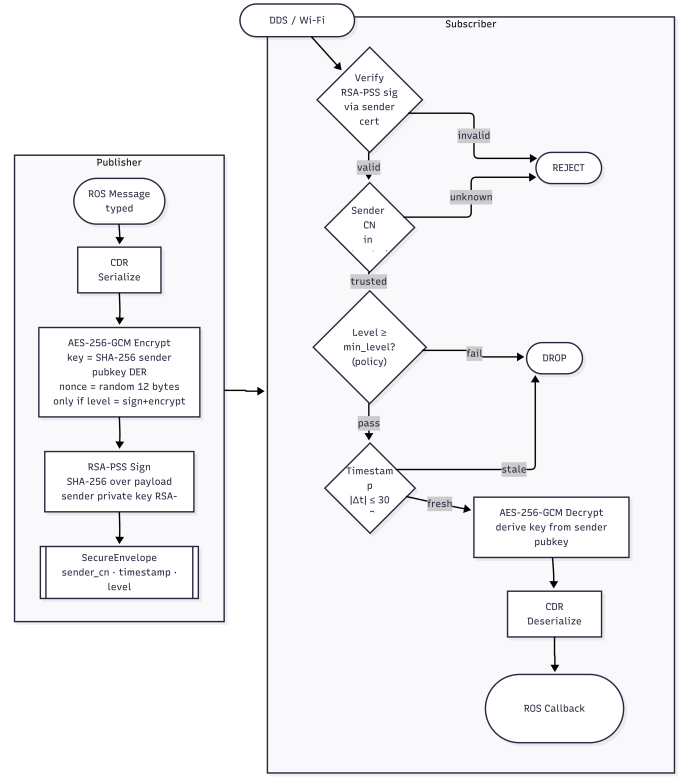


Fig. 20. SecureEnvelope message-protection pipeline. The publisher serializes, optionally AES-256-GCM-encrypts, and RSA-PSS-signs the payload; the subscriber verifies the signature, checks the sender certificate, enforces the `min_level` policy and a 30 s replay window, then decrypts and deserializes. Any failed check rejects or drops the message.

(sign-then-encrypt) and subscribe (verify-before-decrypt) flow is shown in Fig. 20.

#### C. Implementation

Security is added by a `SecureNodeMixin` drop-in; policy resolves by priority (kill switch > explicit code argument > YAML policy > global default). Certificates are minted once per deployment. The deployed secured communication topology is shown in Fig. 21: OptiTrack reaches the robot LAN through a native `optitrack_client`, while the inter-host ROS 2 traffic between the RDK X3, Jetson, and Tello rides FastRTPS DDS over WireGuard-tunnelled Wi-Fi with the `SecureEnvelope` layer applied on top.

#### D. Traceability and Tests

Satisfies FR-7, NFR-10, NFR-11. The middleware is validated against a threat model of seven attack vectors, each mapped to the mechanism that defeats it (Fig. 22) and exercised by the test suite summarized in Table XIII.

1) *Performance benchmark*: The cryptographic cost was measured on the AArch64 embedded platform over 30 timed iterations (after 5 warm-ups) per configuration, in two modes—a *crypto-only* mode isolating wrap/unwrap cost and a full *end-to-end* ROS 2 mode including DDS transport and

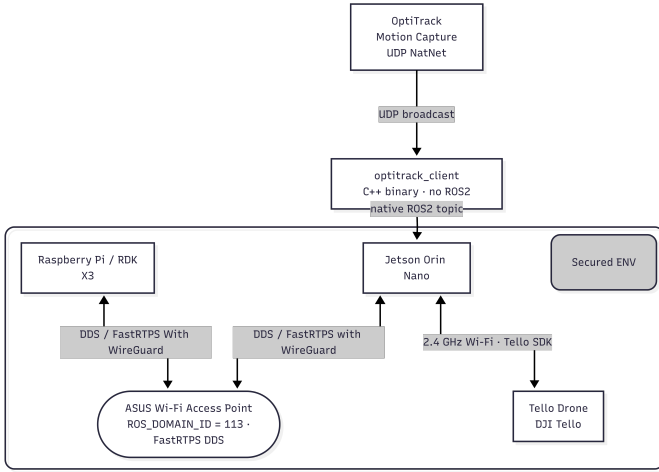


Fig. 21. Secured communication topology: per-link transport (DDS/FastRTPS over WireGuard, Tello SDK) and the secured-environment boundary enclosing the on-robot compute nodes.

TABLE XIII  
SECURITY VALIDATION SUITE

| Test (attack / property)        | Cases | Pass |
|---------------------------------|-------|------|
| Forged signature (spoofing)     | 50    | 50   |
| Bit-flip ciphertext (tampering) | 50    | 50   |
| Replayed envelope (>30 s)       | 40    | 40   |
| Unknown / untrusted sender CN   | 30    | 30   |
| min_level downgrade drop        | 30    | 30   |
| SIGN_ENCRYPT round-trip         | 100   | 100  |
| Kill-switch uniform fallback    | 20    | 20   |

TABLE XIV  
SECURED MESSAGING THROUGHPUT BY LEVEL AND PAYLOAD (MSG/S)

| Level           | 48 B   | 256 B  | 4 KB  | 64 KB |
|-----------------|--------|--------|-------|-------|
| none (baseline) | 12 305 | 10 067 | 2 453 | 176   |
| sign            | 95     | 95     | 91    | 56    |
| sign+encrypt    | 90     | 89     | 82    | 40    |

dispatch—across the three security levels and four payload sizes (Figs. 23–25, Table XIV).

The dominant cost is the RSA-PSS private-key signature: it adds  $\sim 10.4$  ms per message almost independently of payload up to 4 kB (the operation works on a fixed-size digest), accounting for  $\sim 93\%$  of the secured round-trip, while public-key verification is  $\sim 13\times$  cheaper (Fig. 24). Signing therefore collapses throughput from  $\sim 12\,300$  msg/s at the baseline to  $\sim 95$  msg/s—a  $\sim 99\%$  reduction (Table XIV). Adding AES-256-GCM confidentiality on top of signing is comparatively marginal—+6–7% for payloads up to 256 B and +38% only at 64 kB—so sign+encrypt is preferred over sign-only whenever confidentiality is required. The DDS middleware contributes a fixed  $\sim 2.3$  ms to 2.8 ms per round-trip regardless of security level, so the full signed ROS 2 round-trip is  $\sim 13$  ms to 14 ms.

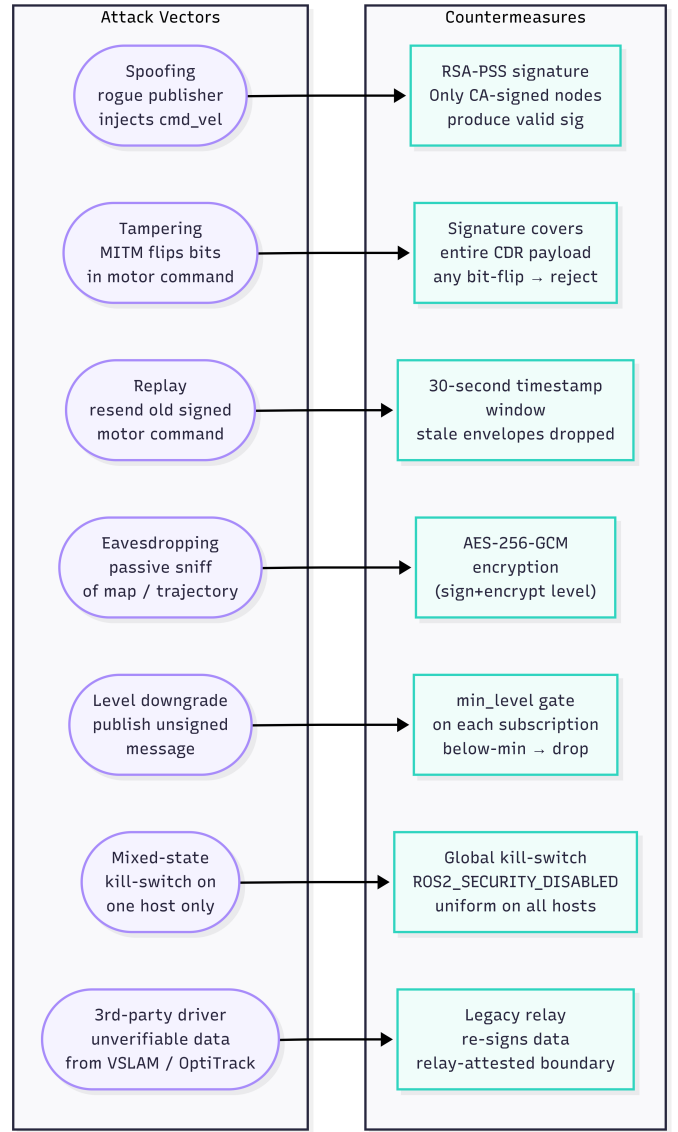


Fig. 22. Threat model: each attack vector (top) is paired with the countermeasure that mitigates it (bottom), spanning spoofing, tampering, replay, eavesdropping, level downgrade, partial kill-switch, and untrusted third-party data.

a) *Real-time implication:* A  $\sim 13$  ms to 14 ms signed round-trip cannot fit a 100 Hz control loop, so security is applied *selectively*: high-rate sensor and control topics run at none, while the lower-rate mission, command, goal, and map topics—which tolerate the added latency—run at sign+encrypt, with the per-node min\_level policy and the global kill switch enforcing the split (NFR-11). The fixed envelope overhead (+296 B signed, +332 B signed-and-encrypted) is below 0.5% above  $\sim 1$  kB payloads and operationally negligible. Identified routes to raising the secured rate—migrating from RSA-PSS to ECDSA(P-256) or Ed25519 (10–30 $\times$  faster signing on ARM), offloading to a TPM/HSM, or pre-signing periodic messages—are left as future work.

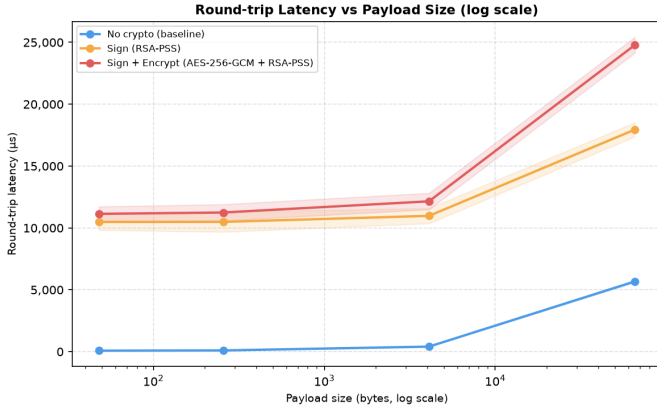


Fig. 23. Crypto-only round-trip latency versus payload size for the three security levels. RSA-PSS signing imposes a large, nearly payload-independent offset; AES-256-GCM adds a comparatively small increment that grows only for large buffers.

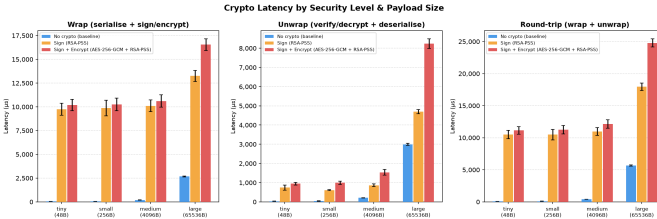


Fig. 24. Latency split into wrap (serialize + sign/encrypt) and unwrap (verify/decrypt + deserialize). Signing on the publisher dominates and is roughly 13 $\times$  costlier than public-key verification on the subscriber.

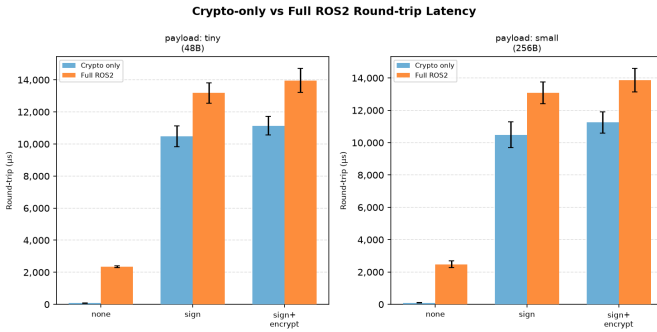


Fig. 25. Crypto-only versus full ROS 2 round-trip for small payloads. The DDS transport and dispatch add a fixed  $\sim 2.3$  ms to 2.8 ms that is independent of, and additive to, the cryptographic cost.

## XVII. EXPERIMENTAL RESULTS

This section consolidates the per-subsystem evaluations already presented in the deep dives into a system-level picture; detailed figures and tables are referenced back rather than repeated.

### A. Experimental Setup

All experiments were run in the  $4 \times 4$  m arena of Fig. 2 (Sec. I) with cardboard-box obstacles and the four corner ArUco wall markers of Sec. IX-B6, plus the goal marker. OptiTrack provides millimetre-accurate ground truth for both

the UAV (used as its positioning source) and the AMR (used only as an external reference for evaluation, never as an AMR input). Results aggregate  $N = 20$  AMR navigation trials and the full survey flights flown to build the maps.

### B. Aerial Mapping and Marker Localization

The stitching pipeline produces a 5 cm/cell occupancy map of the arena from the UAV survey, with the deployed similarity-transform baseline and the learned quality gate accepting or rejecting each map (Sec. VII). Marker poses are recovered through the mirror-rig PnP chain and robust aggregation of Sec. VIII.

### C. State Estimation Accuracy

Against OptiTrack ground truth, the from-scratch EKF holds a mean absolute position error of 4.6 cm in  $x$  and 6.3 cm in  $y$  (RMS 7.3 cm/8.5 cm), versus 52.3 cm/41.6 cm for raw wheel odometry—an  $\sim 88\%$  reduction in mean error from fusing IMU, gated cuVSLAM velocity, and ArUco fixes (Figs. 14–15). The adaptive VIO gate replaced 211 loop-closure spike samples, suppressing impulses of up to  $4.9 \text{ m s}^{-1}$  (Figs. 12–13). On the aerial side, the UAV tracked its survey trajectory with mean absolute error  $\leq 3.8$  cm per axis (Table VIII).

### D. Navigation Performance

Across the  $N = 20$  trials the AMR reached the goal marker in 17/20 runs (85%) with no collisions, at a mean cross-track error of 6.1 cm and a mean mission time of 74 s from map handoff to arrival. The three failures occurred under early-mission map sparsity, where planning stalled past the run's time budget; the autonomous replanning trigger handled all other transient blockages without operator intervention. When the aerial map was withheld to force the fallback, frontier exploration located and homed onto the goal autonomously (Sec. XIV); quantitative coverage-versus-time results are pending.

### E. Safety and Security Validation

The emergency-braking layer stopped the robot in 30/30 approach trials with a mean response time of 0.34 s—matching the 5/15 Hz streak-detector bound—and produced no false triggers in obstacle-free traversal (Table XII). The security middleware passed its functional suite against all seven modelled attack vectors (Table XIII); its measured cost is dominated by RSA-PSS signing at  $\sim 10.4$  ms per message ( $\sim 99\%$  throughput reduction), with AES-256-GCM adding only 6–7% for sub-kilobyte payloads, motivating the selective, per-topic application of security (Sec. XVI).

### F. Discussion

The results support the central hypothesis: the aerial prior plus onboard fusion yields reliable GPS-denied navigation, and the system degrades gracefully—to frontier exploration when the map is rejected, and to dead-reckoning when ArUco fixes are unavailable. The principal limitations are the unbounded position drift between marker sightings, a static IMU gyro-bias model, the sequential (rather than joint) EKF update, and the

signing latency that confines authenticated messaging to lower-rate topics. The use of OptiTrack as a GNSS surrogate for the UAV is a testbed convenience; a field deployment would substitute GNSS or an onboard georeferencing method without changing the cooperative architecture.

## XVIII. CONCLUSION AND FUTURE WORK

We presented a cooperative UAV-AMR system that navigates to a goal in a GPS-denied, initially unmapped arena by having an aerially positioned drone build the map and localize the goal, then handing that prior to a ground robot that fuses it with onboard sensing. The design satisfies the project's functional and non-functional requirements through a from-scratch EKF, a similarity-transform stitching pipeline with embedded deep learning, robust marker localization, registration-free map fusion, a frontier-exploration fallback, an independent emergency-braking layer, and an application-layer security middleware, all integrated in ROS 2 across three networked platforms. In our testbed the EKF cut mean position error by  $\sim 88\%$  versus wheel odometry, the AMR reached the goal in 85% of trials without collision, emergency braking responded in  $\sim 0.34$ s, and benchmarking the security layer exposed RSA-PSS signing ( $\sim 10.4$  ms/message) as the binding constraint that shapes its selective deployment. Future work includes more frequent ArUco re-anchoring to bound dead-reckoning drift, online gyro-bias estimation, dynamic-obstacle prediction in the planner, faster signatures (ECDSA/Ed25519) for higher-rate authenticated topics, and scaling to larger multi-robot teams.

## ACKNOWLEDGMENT

The authors thank the teams at OMRON Automation and Nuclea Solutions for their feedback throughout the project and for their insight into implementing the system to a high standard of quality. The authors also acknowledge the use of Claude (Anthropic) as an assistant during development and documentation.

## REFERENCES

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, 2022.
- [2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.
- [3] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Trans. ASME J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960.
- [4] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, 2004.
- [5] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [6] P. J. Burt and E. H. Adelson, "A multiresolution spline with application to image mosaics," *ACM Trans. Graph.*, vol. 2, no. 4, pp. 217–236, 1983.
- [7] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognit.*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [8] T. Collins and A. Bartoli, "Infinitesimal plane-based pose estimation," *Int. J. Comput. Vis.*, vol. 109, no. 3, pp. 252–286, 2014.

- [9] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [10] D. DeTone, T. Malisiewicz, and A. Rabinovich, "SuperPoint: Self-supervised interest point detection and description," in *Proc. IEEE/CVF CVPR Workshops*, 2018, pp. 224–236.
- [11] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local feature matching at light speed," in *Proc. IEEE/CVF ICCV*, 2023, pp. 17581–17592.
- [12] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.
- [13] E. Weiszfeld, "Sur le point pour lequel la somme des distances de  $n$  points donnés est minimum," *Tôhoku Math. J.*, vol. 43, pp. 355–386, 1937.
- [14] M. J. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, 2007.
- [15] P. Tokekar, J. Vander Hook, D. Mulla, and V. Isler, "Sensor planning for a symbiotic UAV and UGV system for precision agriculture," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1498–1511, 2016.
- [16] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE Int. Symp. Comput. Intell. Robot. Autom. (CIRA)*, 1997, pp. 146–151.
- [17] V. DiLuoffo, W. R. Michalson, and B. Sunar, "Robot Operating System 2: The need for a holistic security approach to robotic architectures," *Int. J. Adv. Robot. Syst.*, vol. 15, no. 3, 2018.
- [18] A. Korovko, D. Slepichev, A. Klyuchnikov, K. Chebukov, *et al.*, "cu-VSLAM: CUDA accelerated visual odometry and mapping," NVIDIA, 2025. [Online]. Available: <https://arxiv.org/abs/2506.04359>
- [19] NVIDIA, "Isaac ROS Visual SLAM (isaac\_ros\_visual\_slam)," NVIDIA Isaac ROS documentation, 2024. [Online]. Available: [https://github.com/NVIDIA-ISAAC-ROS/isaac\\_ros\\_visual\\_slam](https://github.com/NVIDIA-ISAAC-ROS/isaac_ros_visual_slam)
- [20] D. Ortiz *et al.*, "collab\_nav\_ground-jetson: Jetson navigation and aerial-mapping stack," GitHub, 2026. [Online]. Available: [https://github.com/David-OC17/collab\\_nav\\_ground-jetson](https://github.com/David-OC17/collab_nav_ground-jetson)
- [21] N. Rosales *et al.*, "collab\_nav\_ground-rasp: RDK X3 drive, odometry, and EKF stack," GitHub, 2026. [Online]. Available: [https://github.com/NoeRos22/collab\\_nav\\_ground-rasp](https://github.com/NoeRos22/collab_nav_ground-rasp)
- [22] A. Romo *et al.*, "security\_middleware: Application-layer ROS 2 security middleware," GitHub, 2026. [Online]. Available: [https://github.com/FrediRomo/security\\_middleware](https://github.com/FrediRomo/security_middleware)